EPISEN École Publique d'Ingénieurs de la Santé Et du Numérique

UPEC Connaissance - Action
UNIVERSITÉ PARIS-EST CRÉTEIL VAL DE MARNE

"Que la force soit avec toi"

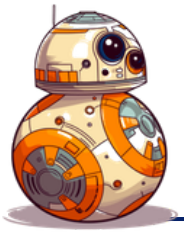
COMPTE RENDU DU GROUPE STAR WARS

OLIVIER MAXENCE

AUGER VALENTIN

AL BYDEOI OTHMEN





Introduction : Présentation du projet

Ce projet s'inscrit dans le cadre de l'ECUE 325 — Systèmes de mesures nomades à l'EPISEN. L'objectif principal est de concevoir, développer et implémenter une chaîne complète de simulation et de traitement du signal sous MATLAB, englobant la génération de signaux complexes, la simulation de déformations (linéaires et non linéaires), leur analyse spectrale par Transformée de Fourier Rapide (FFT), ainsi que la restauration du signal d'origine.

Pour rendre cet outil interactif, ergonomique et visuel, l'ensemble des modules d'analyse et de traitement a été intégré au sein d'une Interface Homme-Machine (IHM) développée via MATLAB App Designer.

Ce compte rendu détaille la démarche suivie à travers les sections suivantes :

- Les capteurs et la chaîne de mesure : Étude théorique, caractéristiques métrologiques et modélisation de l'acquisition.
- Génération et déformation des signaux : Formulation mathématique et implémentation sous MATLAB des altérations linéaires (gain, offset) et non linéaires (exponentielle, logarithme).
- Analyse spectrale, estimation et correction : Utilisation de la FFT pour identifier les paramètres d'erreur et reconstruire le signal d'origine.
- Conception de l'IHM : Présentation des fonctionnalités de l'interface graphique



SOMMAIRE :

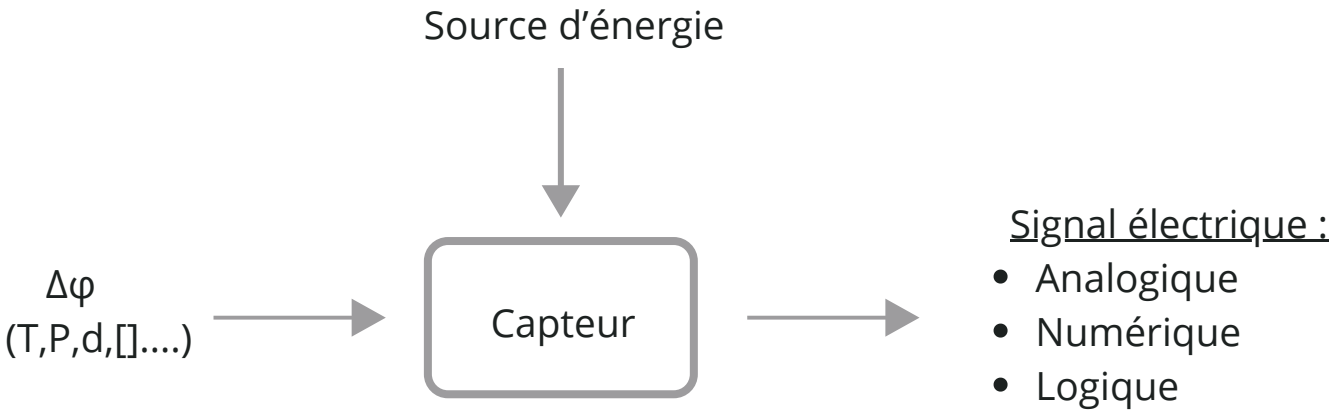
- 1. Les capteurs ..... page 4
- 1.1. Définition et principe général
- 1.2. Place du capteur dans la chaîne de mesure
- 1.3. Caractéristiques métrologiques d'un capteur
- 2. Génération du signal ..... page 7
- 2.1. Équation du signal de référence
- 2.2. Implémentation dans le script MATLAB (gene-signal.m)
- 3. Les Déformations ..... page 8
- 3.1. Erreurs linéaires
- 3.2. Erreurs non linéaires
- 3.3. Principe de la correction
- 4. Analyse spectrale par FFT ..... page 11
- 4.1. Principe théorique
- 4.2. Traitement du spectre sous MATLAB
- 5. Estimation et Correction ..... page 12
- 5.1. Principe théorique : Séparation du signal et centrage
- 5.2. Cas 1 : Estimation et Correction de l'OFFSET
- 5.3. Cas 2 : Estimation et Correction du GAIN (
- 5.4. Cas 3 et 4 : Combinaisons GAIN + OFFSET
- 6. Codage des fonctions ..... page 15
- 6.1. Script de génération du signal
- 6.2. Script de la déformation .
- 6.3. Script de la FFT
- 6.4. Script de la correction
- 7. L'IHM ..... page 29
- 7.1. Page d'accueil
- 7.2. Page Menu Signal
- 7.3. Page Menu Erreur
- 7.4. Page Menu FFT
- 7.5. Page Menu Estimation
- 7.6. Page Menu Correction
- 8. Bonus ..... page 38
- 8.1. Page Menu Option (Dark Debug et exportations)
- 9. Conclusion ..... page 42
- Documentation annexe ..... page 43



1.Les capteurs

1.1. Définition et principe général

Un capteur est un élément de l'interfaçage d'un système mesurant une grandeur physique et la convertissant en un signal électrique exploitable. En fonction du modèle et du besoin, la mesure nécessite une source d'énergie externe et fournit un signal.



1.2. Place du capteur dans la chaîne de mesure

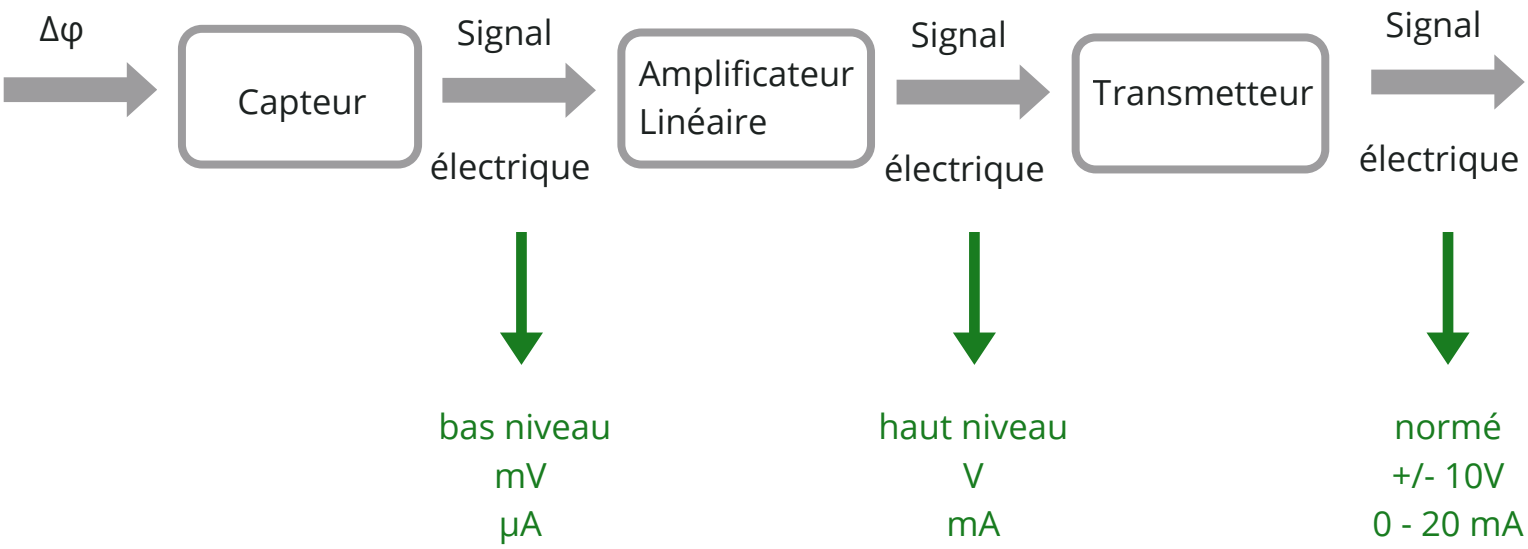
La chaîne d'acquisition et de traitement de l'information suit plusieurs étapes successives :

- **Capteur / Transducteur :** conversion de la grandeur physique en un signal électrique de bas niveau.
- **Pré-traitement / Amplification linéaire :** mise en forme et élévation du signal à un haut niveau.
- **Transmission / Transmetteur :** filtrage, conversion A/N ou décodage pour obtenir un signal normé
- **Traitement du signal :** exploitation numérique des données.

Action ou affichage : restitution de l'information (écran, voyant, alarme ou actionneur).

Note : Les étapes de pré-traitement et de traitement direct du signal sont fréquemment intégrées directement au sein du corps du capteur

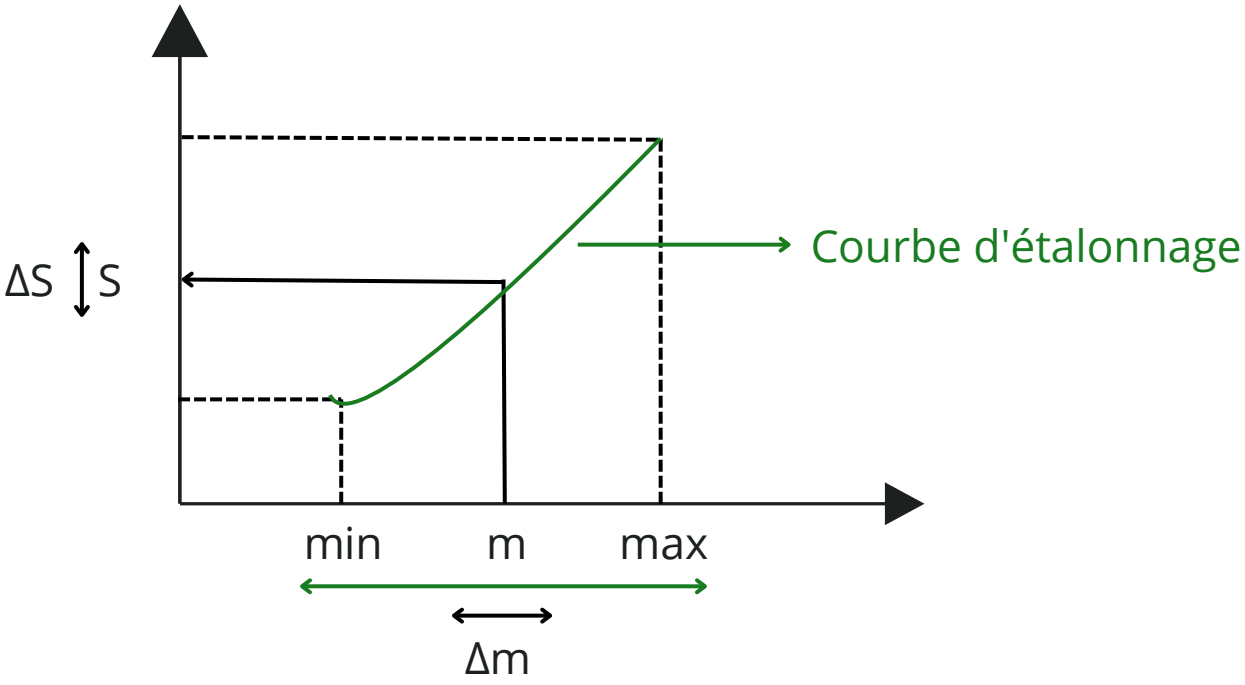
Schéma de la chaîne d'acquisition et de traitement de l'information



1.3. Caractéristiques métrologiques d'un capteur

- **Étendue de mesure :** plage des valeurs de la grandeur physique mesurable (du minimum au maximum).
- **Résolution** : plus petite variation de la grandeur d'entrée détectable par le capteur.
- **Sensibilité (S) :** rapport entre la variation du signal de sortie ΔS et la variation de la grandeur d'entrée.
- **Précision :** capacité du capteur à délivrer une valeur proche de la valeur réelle.
- **Rapidité :** temps de réponse du capteur à une variation brutale de la grandeur mesurée.
- **Linéarité :** régularité de la sensibilité sur toute l'étendue de mesure (la sensibilité S doit être la plus constante possible quel que soit le point de mesure m).

Schéma de la Courbe d'étalonnage



2.Génération du signal

2.1. Équation du signal de référence

Le signal de référence s(t) est composé d'une constante continue et d'une somme de trois harmoniques sinusoïdaux. Sa formulation mathématique temporelle s'écrit :

$$S_{init} = M + \sum_{i=1}^{n} Ai \times sin\left(2\pi fit\right)$$

Avec n = 3 dans notre cas d'étude (correspondant au nombre d'harmoniques).
Description des variables et paramètres :

- M : composante continue ou valeur moyenne du signal.
- Ai : amplitude du ième harmonique (A1, A2, A3).
- fi : fréquence de la ième harmonique (f1, f2, f3).
- t : variable temporelle discrétisée à partir de la fréquence d'échantillonnage fe

2.2. Implémentation dans le script MATLAB (gene-signal.m)

Cette modélisation mathématique sert de base au développement du script algorithmique gene-signal.m. La fonction est structurée avec le prototype suivant :

[s] = gene_signal(M, Ai, Fi, Fe, AFF)

Rôle des arguments d'entrée :

- M : offset du signal.
- Ai : vecteur contenant les amplitudes [A1, A2, A3].
- Fi : vecteur contenant les fréquences [f1, f2, f3].
- Fe : fréquence d'échantillonnage permettant de générer le vecteur temps t.
- AFF : paramètre booléen de contrôle d'affichage (0 pour désactiver, 1 pour afficher).

3. Les Déformations

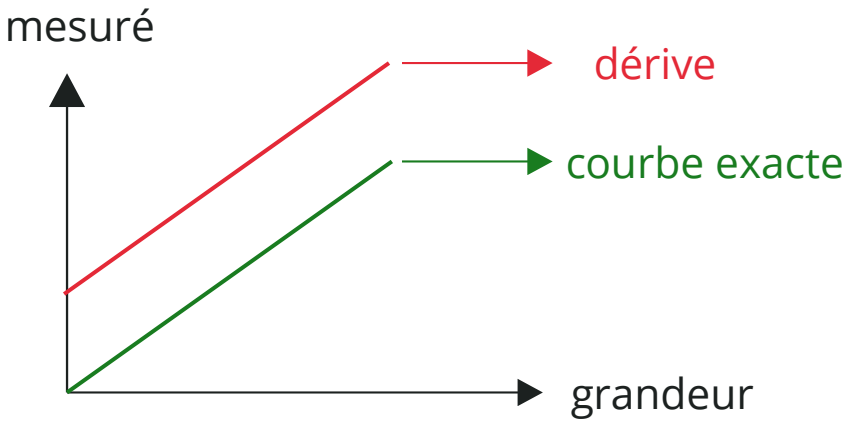
Lors de l'acquisition et de la transmission d'une mesure, la réponse du capteur subit des altérations par rapport à la valeur réelle. Ces déformations s'classent en deux catégories majeures : les erreurs linéaires et les erreurs non linéaires.

3.1. Erreurs linéaires

Erreur d'offset (erreur de zéro / constante) :

Il s'agit d'un décalage constant appliqué sur toute l'étendue de mesure. La pente de la courbe reste identique à la caractéristique exacte, mais la réponse du capteur présente un écart initial dès l'origine.

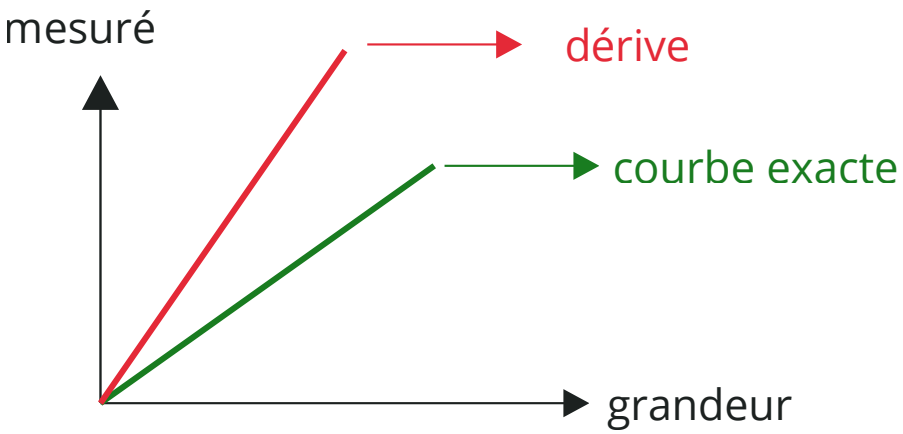
$$\text{Expression} : G \cdot y_i + \text{OFF}$$



Erreur de gain (erreur d'échelle / multiplicative) :

Cette erreur se caractérise par une modification de la pente de la droite de réponse. L'écart entre la valeur réelle et la valeur mesurée augmente progressivement avec l'amplitude de la grandeur ou au cours du temps.

$$\text{Expression} : G \cdot y_i + G \cdot \text{OFF}$$

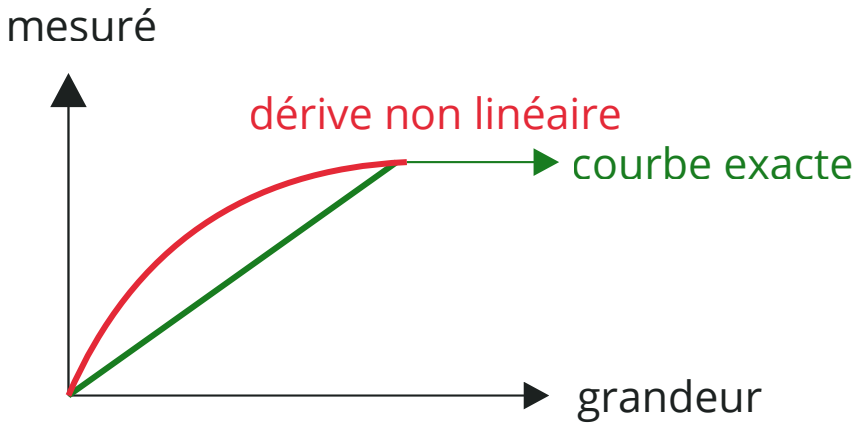


3.2. Erreurs non linéaires

Les erreurs non linéaires déforment la réponse du capteur de manière non proportionnelle, rendant la réponse non rectiligne.

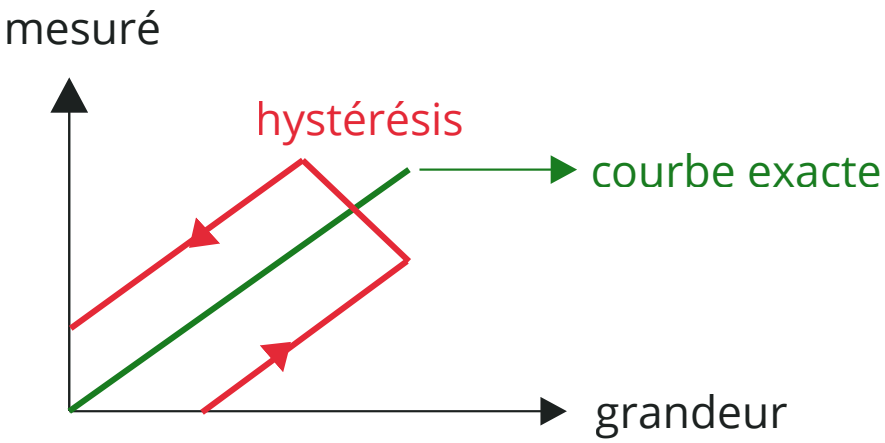
Erreur de linéarité (dérive non linéaire) :

La caractéristique du capteur s'écarte de la droite théorique pour prendre une forme courbe. La sensibilité du capteur ne reste plus constante sur l'ensemble de la plage de mesure.



Erreur d'hystérésis :

La valeur de sortie dépend du sens de variation de la grandeur d'entrée (selon que la grandeur est en phase croissante ou décroissante). Le trajet suivi lors de la montée diffère de celui de la descente, créant un cycle d'hystérésis.



3.3. Principe de la correction

Pour retrouver le signal d'origine à partir du signal déformé, le système doit appliquer la fonction réciproque de la déformation subie :

$$f^{-1} \circ f = \mathrm{Id}$$

f représente le modèle de déformation introduit par le système de mesure.

f^{-1} est l'opérateur de correction appliqué par le traitement algorithmique.

Id correspond au signal initial parfaitement restauré.

4. Analyse spectrale par FFT

4.1 Principe théorique

La Transformée de Fourier permet de transposer un signal du domaine temporel vers le domaine fréquentiel. Dans un système numérique, on utilise la Transformée de Fourier Discrète (TFD) via l'algorithme rapide FFT (Fast Fourier Transform).

L'analyse spectrale permet d'identifier :

- La composante continue 0Hz : Elle représente la valeur moyenne du signal.
- La fréquence fondamentale : Il s'agit de la fréquence du harmonique principal constituant le signal d'origine.
- Les harmoniques parasites : Fréquences induites par les déformations non linéaires subies par le signal.

4.2 Traitement du spectre sous MATLAB

Pour obtenir une représentation fréquentielle exploitable sur l'IHM, deux propriétés essentielles de la FFT sont appliquées :

Symétrie spectrale et spectre monolatéral :
Pour tout signal réel, la FFT génère un spectre symétrique (effet miroir) par rapport à la fréquence de Nyquist fe/2. La seconde moitié du spectre (allant de fe / 2 à fe n'apporte aucune information supplémentaire et contient les réplicats symétriques.
Pour la représentation graphique, on extrait uniquement la première moitié du spectre (de 0 à fe / 2). Afin d'éviter les erreurs d'indexation dans le tableau lorsque le nombre d'échantillons N est impair, le nombre de points conservé est arrondi à l'entier inférieur grâce à la fonction floor(N/2).

Construction de l'axe des fréquences :
La FFT retourne un tableau de points discret. Pour convertir ces indices en fréquences réelles exprimées en Hertz , la résolution fréquentielle est $\Delta f = \frac{f_e}{N}$ appliquée à chaque point k du spectre :

$$f(k) = k \cdot \frac{f_e}{N} \quad \text{pour } k \in [0, N_2 - 1]$$

5.Estimation et Correction

L'objectif de cette méthode est d'utiliser le domaine fréquentiel (spectre FFT) pour estimer les paramètres d'erreur linéaire (P0 pour l'Offset et P1 pour le Gain) et reconstruire le signal d'origine.

5.1. Principe théorique : Séparation du signal et centrage

L'objectif de cette méthode est d'utiliser le domaine fréquentiel (spectre FFT) pour estimer les paramètres d'erreur linéaire (P0 pour l'Offset et P1 pour le Gain) et reconstruire le signal d'origine.

$$s_{base}(t) = s(t) - \langle s(t) \rangle$$

- À 0 Hz (composante continue) : L'amplitude du spectre |S(0)| représente directement la valeur moyenne du signal. Comme le signal est centré, sa valeur initiale à 0 Hz vaut strictment 0.

- À la fréquence fondamentale (0H z) : L'amplitude du spectre |S(f fond)| représente l'amplitude harmonique du signal.

5.2. Cas 1 : Estimation et Correction de l'OFFSET (P0)

Lorsqu'un Offset P_0 est ajouté au signal centré $s_{base}(t)$:

$$s_{deforme}(t) = s_{base}(t) + P_0$$

L'offset P_0 est une constante temporelle, ce qui correspond mathématiquement à une composante purement continue à 0 Hz dans le spectre.

- Mesure : On lit directement l'amplitude du premier point du spectre de la déformée (0Hz) :

$$P_{0,mesure} = |S_{deforme}(0)|$$

La correction s'effectue en soustrayant la valeur mesurée au signal déformé :

$$s_{corrige}(t) = s_{deforme}(t) - P_{0,mesure}$$

5.3. Cas 2 : Estimation et Correction du GAIN (P1)

Un gain P1 multiplie l'ensemble du signal. Pour observer facilement son effet à la fois sur la composante continue et sur l'amplitude :

On prend le signal centré et on lui ajoute une composante continue connue égale à 1 :

$$s_{base}(t) = (s(t) - \langle s(t) \rangle) + 1$$

On applique la déformation par le gain P1

$$s_{deforme}(t) = P_1 \cdot s_{base}(t)$$

À 0 Hz, la composante continue de s_{base} valait 1. Après déformation, elle vaut 1 × P1 = P1.

Mesure : On calcule le rapport entre la composante continue du signal déformé et celle du signal de base :

$$P_{1,mesure} = \frac{|S_{deforme}(0)|}{|S_{base}(0)|}$$

La correction s'effectue par division directe par le gain estimé :

$$s_{corrige}(t) = \frac{s_{deforme}(t)}{P_{1,mesure}}$$

5.4. Cas 3 & 4 : Combinaisons GAIN + OFFSET

Lorsque le signal subit simultanément un Gain et un Offset, les deux composantes interagissent sur la valeur à 0 Hz. On procède alors en deux étapes :

Estimation du Gain (P1) : Le Gain modifie de la même façon l'amplitude à toutes les fréquences. On mesure le rapport des amplitudes sur le pic de la fréquence fondamentale (f > 0 Hz) où l'offset n'a aucune influence :

On applique la déformation par le gain P1

$$P_{1,mesure} = \frac{|S_{deforme}(f_{fond})|}{|S_{base}(f_{fond})|}$$

Ordre GAIN puis OFFSET : $s_{deforme} = P_1 \cdot s_{base} + P_0$

$$P_{0,mesure} = |S_{deforme}(0)| - (1 \cdot P_{1,mesure})$$

$$\text{Correction : } s_{corrige}(t) = \frac{s_{deforme}(t) - P_{0,mesure}}{P_{1,mesure}}$$

Ordre OFFSET puis GAIN : $s_{deforme} = P_1 \cdot (s_{base} + P_0)$

$$P_{0,mesure} = \left(\frac{|S_{deforme}(0)|}{P_{1,mesure}} \right) - 1$$

$$\text{Correction : } s_{corrige}(t) = \frac{s_{deforme}(t)}{P_{1,mesure}} - P_{0,mesure}$$

6. Codage des fonction

6.1 script de génération du signal

La fonction gene_signal génère un signal temporel en combinant une valeur moyenne M et plusieurs harmoniques sinusoïdales définies par leurs amplitudes Ai et leurs fréquences Fi, avec une option d'affichage graphique (AFF).

La durée est fixée à 1 seconde (T = 1) par choix de conception standard : cela permet aux fréquences d'indiquer directement le nombre de cycles par seconde, tout en restant une valeur arbitraire et facilement modifiable selon les besoins de la simulation.

```
4   function [s] = gene_signal(M, Ai, Fi, fe, AFF)
5   % Durée du signal : 1 seconde
6   T = 1;
7
8   % Vecteur temps (N échantillons sur 1 seconde)
9   t = 0 : 1/fe : T - 1/fe;
10
11  % Calcul de la somme des harmoniques + la valeur moyenne M
12  % Note : sum() permet d'éviter la boucle 'for' en MATLAB
13  s = M + sum(Ai(:) .* sin(2 * pi * Fi(:) * t), 1);
14
15  % Affichage
16  if AFF == 1
17      plot(t, s);
18      grid on;
19      xlabel('Temps (s)');
20      ylabel('Amplitude');
21      title('Signal généré');
22  end
23  end
```

Définition de la fonction et des paramètres d'entrée

La fonction prend les arguments suivants :

- M : La valeur moyenne (ou composante continue) du signal.
- Ai : Le vecteur contenant les amplitudes des différentes composantes sinusoïdales (harmoniques).
- Fi : Le vecteur contenant les fréquences (en Hertz) de ces mêmes composantes.
- fe : La fréquence d'échantillonnage.
- AFF : Un drapeau (flag) d'affichage (si AFF == 1, le graphique du signal est tracé).

Calcul du signal

- Le signal s est calculé en additionnant la valeur moyenne M et la somme des fonctions sinusoïdales.
- L'utilisation de la fonction sum() permet d'éviter l'utilisation d'une boucle for (ce qui optimise les performances sous MATLAB) en effectuant un calcul vectorisé sur les amplitudes Ai et les fréquences Fi.

6.2 script de la déformation

La fonction deform_signal permet de modifier ou de transformer un signal d'entrée s en lui appliquant différents types de traitements mathématiques selon un mode choisi (choix), à l'aide de paramètres de réglage P0 et P1

Ce que fait concrètement cette fonction :
Elle utilise une structure de choix multiples (switch) pour appliquer l'une des 6 déformations suivantes :

- Cas 1 (Offset) : Ajoute une valeur constante P0 au signal (sd = s + P0).
- Cas 2 (Gain) : Multiplie le signal par un facteur multiplicatif P1 (sd = P1 * s).
- Cas 3 (Gain + Offset) : Multiplie d'abord le signal par P1 puis ajoute P0 (sd = P1 * s + P0).
- Cas 4 (Offset + Gain) : Ajoute d'abord P0 au signal avant de multiplier le tout par P1 (sd = P1 * (s + P0)). .

- Cas 5 (Exponentielle) : Applique une transformation exponentielle modifiée par les paramètres P0 et P1 (sd = P0 * exp(P1 * s)).
- Cas 6 (Logarithme) : Applique une transformation logarithmique contrôlée par P1 et P0 (sd = P1 * log(s) + P0).
- Autrement (otherwise) : Si le choix ne correspond à aucun cas valide, le signal initial est retourné sans modification (sd = s).

```
function sd = deform_signal(s, choix, P0, P1)
    switch choix
        case 1
            % Offset (OFFSET)
            sd = s + P0;
        case 2
            % Gain
            sd = P1 * s;
        case 3
            % Gain + Offset
            sd = P1 * s + P0;
        case 4
            % Offset + Gain
            sd = P1 * (s + P0);
        case 5
            % Exponentielle
            sd = P0 * exp(P1 * s);
        case 6
            % Logarithme
            sd = P1 * log(s) + P0;
        otherwise
            sd = s;
    end
end
```

6.3 script de la FFT

La fonction spectre permet de calculer et de tracer le spectre fréquentiel d'un signal temporel en utilisant la Transformée de Fourier Rapide (FFT), tout en déterminant automatiquement sa fréquence fondamentale.

```
function f_fond = spectre(ax, s, fe)
    %  Nombre d'échantillons du signal (valeur entiere)
    N = length(s);

    %  Calcul de la Transformée de Fourier Rapide (FFT)
    S = abs(fft(s));

    %  Découpe de la moitié du spectre (Symétrie de la FFT)
    % IMPORTANT : N/2 peut donner un nombre décimal (float) si N est impair.
    % La fonction floor() force le résultat à devenir un ENTIER STRICT (arrondi inférieur).
    % MATLAB exige un entier car un indice de tableau/case mémoire ne peut pas être un float.
    N2 = floor(N/2);

    % Extraction des N2 premiers points (indexation de 1 à l'entier N2)
    S = S(1:N2);

    %  Construction de l'axe des fréquences réelles (en Hz)
    f = (0:N2-1) * fe / N;

    %  Affichage du graphique dans l'axe de l'IHM
    stem(ax, f, S);
    grid(ax, 'on');
    xlabel(ax, 'Fréquence (Hz)');
    ylabel(ax, '|S(f)|');
    title(ax, 'Spectre du signal');

    %  Recherche de la fréquence fondamentale
    if length(S) > 1
        % max() cherche l'amplitude maximale
        % idx renvoie l'INDICE ENTIER de la position du pic dans le tableau
        [~, idx] = max(S(2:end)); % On saute le 1er indice (0 Hz / composante continue)

        % Décalage de +1 sur l'indice car on a commencé la recherche à partir de S(2)
        f_fond = f(idx + 1);
    else
        f_fond = 0;
    end
```

Définition de la fonction et des paramètres

La fonction prend en entrée et en sortie les éléments suivants :

- ax : L'axe graphique cible (pour une IHM) où sera tracé le spectre.
- s : Le signal temporel à analyser.
- fe : La fréquence d'échantillonnage du signal.
- f_fond (Sortie) : La fréquence fondamentale du signal.

Ce que fait la fonction :

- Calcul de la FFT : Elle calcule la longueur du signal N et applique la Transformée de Fourier Rapide via abs(fft(s)) pour obtenir l'amplitude spectrale.
- Symétrie et découpage : Elle ne conserve que la première moitié du spectre (jusqu'à N2 = floor(N/2)) pour éliminer la redondance symétrique.
- Axe des fréquences : Elle reconstruit l'axe des fréquences réelles en Hertz (f) en fonction de la fréquence d'échantillonnage et du nombre de points.
- Affichage graphique : Elle trace le spectre à l'aide de la fonction stem(ax, f, S) sur l'axe indiqué, avec une grille, des titres et des libellés d'axes.
- Recherche de la fondamentale : Elle cherche l'amplitude maximale du spectre (en ignorant la composante continue à 0 Hz) pour identifier et retourner la fréquence fondamentale (f_fond) du signal.

6.4 script de la correction

La fonction correcteOFFSET génère un signal de base, lui applique une déformation de type offset, mesure cet offset dans le domaine fréquentiel, puis affiche et compare graphiquement les spectres du signal initial et du signal déformé.

6.4.1 OFFSET

La fonction correcteOFFSET génère un signal de base, lui applique une déformation de type offset, mesure cet offset dans le domaine fréquentiel, puis affiche et compare graphiquement les spectres du signal initial et du signal déformé

Ce que fait concrètement cette fonction :

- Génération et centrage : Elle génère un signal à l'aide de gene_signal puis le centre (pour obtenir une moyenne nulle) afin de former le signal de référence s_base.

Application de la déformation : Elle applique une déformation par offset (choix 1) sur s_base via la fonction deform_signal pour obtenir s_deforme.

- Analyse spectrale et mesure : Elle calcule la Transformée de Fourier Rapide (FFT) normalisée des deux signaux et mesure directement l'offset introduit en isolant la composante continue à 0 Hz (offset_mesure = S_deforme(1)).
- Visualisation graphique : Si un axe d'affichage est fourni, elle superpose le spectre du signal déformé (en rouge) et celui du signal de base (en bleu pointillé), accompagnés d'une grille, d'un titre et d'une légende.

```
function [s_base, s_deforme, s_corrige, offset_mesure] = correcteOFFSET(M, Ai, Fi, fe, P0, P1, ax_spectre, ax_correction)
    %  Génération du signal de base
    s_temp = gene_signal(M, Ai, Fi, fe, 0);

    % Signal centré (moyenne nulle : amplitude nulle à 0 Hz)
    s_base = s_temp - mean(s_temp);

    %  Application de la déformation OFFSET (choix 1)
    s_deforme = deform_signal(s_base, 1, P0, P1);

    %  Calcul de la FFT
    N = length(s_deforme);
    S_deforme = abs(fft(s_deforme)) / N;
    S_base    = abs(fft(s_base)) / N;

    offset_mesure = S_deforme(1);

    %  Tracer les deux spectres
    if nargin >= 7 && ~isempty(ax_spectre)
        N2 = floor(N/2);
        f = (0:N2-1) * fe / N;

        hold(ax_spectre, 'off');

        % Signal déformé (en rouge)
        stem(ax_spectre, f, S_deforme(1:N2), 'r', 'LineWidth', 2, 'DisplayName', 'Signal Déformé');
        hold(ax_spectre, 'on');

        % Signal de base (en bleu pointillé légèrement décalé pour rester visible)
        stem(ax_spectre, f + 0.1, S_base(1:N2), 'b--', 'LineWidth', 1.2, 'DisplayName', 's_i - \langle s_i \rangle');

        grid(ax_spectre, 'on');
        xlabel(ax_spectre, 'Fréquence (Hz)');
        ylabel(ax_spectre, '|S(f)|');
        title(ax_spectre, 'Spectre : de la déformée et de s_i - \langle s_i \rangle');
        legend(ax_spectre, 'show');
    end
```

6.4.2 GAIN

La fonction correcteGAIN génère un signal de base adapté, lui applique une déformation de type gain (multiplication), estime la valeur de ce gain à partir du domaine fréquentiel, puis trace et compare graphiquement les spectres des signaux.

```
function [s_base, s_deforme, s_corrige, gain_mesure] = correcteGAIN(M, Ai, Fi, fe, P0, P1, ax_spectre, ax_correction)
    %  Génération du signal de base
    s_temp = gene_signal(M, Ai, Fi, fe, 0);

    % Centrage du signal + ajout de +1 pour avoir une composante à 0 Hz
    s_base = (s_temp - mean(s_temp)) + 1;

    %  Application de la déformation GAIN (choix 2 = Gain P1)
    choix_deformation = 2;
    s_deforme = deform_signal(s_base, choix_deformation, P0, P1);

    %  Calcul de la FFT
    N = length(s_deforme);
    S_deforme = abs(fft(s_deforme)) / N;
    S_base    = abs(fft(s_base)) / N;

    %  Estimation du Gain à partir de la composante à 0 Hz (ou sur le pic)
    % À 0 Hz, l'amplitude vaut 1 * P1 = P1
    gain_mesure = S_deforme(1) / S_base(1);

    %  Affichage des spectres sur l'axe de l'IHM (Menu Estimation)
    if nargin >= 7 && ~isempty(ax_spectre)
        N2 = floor(N/2);
        f = (0:N2-1) * fe / N;

        hold(ax_spectre, 'off');

        % Signal déformé avec le Gain P1 (en rouge)
        stem(ax_spectre, f, S_deforme(1:N2), 'r', 'LineWidth', 2, 'DisplayName', 'Signal Déformé (x P_1)');
        hold(ax_spectre, 'on');

        % Signal de base s_i - <s_i> + 1 (en bleu pointillé légèrement décalé)
        stem(ax_spectre, f + 0.1, S_base(1:N2), 'b--', 'LineWidth', 1.2, 'DisplayName', 's_i - \langle s_i \rangle + 1');

        grid(ax_spectre, 'on');
        xlabel(ax_spectre, 'Fréquence (Hz)');
        ylabel(ax_spectre, '|S(f)|');
        title(ax_spectre, 'Spectre : Gain sur (s_i - \langle s_i \rangle + 1)');
        legend(ax_spectre, 'show');
    end
```

Ce que fait la fonction :

- Génération et préparation du signal : Elle génère d'abord un signal temporaire via gene_signal, puis le centre et ajoute une constante (+1) afin de lui garantir une composante connue à 0 Hz pour former le signal de base s_base.
- Application de la déformation : Elle applique un gain (choix 2) à s_base à l'aide de la fonction deform_signal avec le facteur P1 pour obtenir s_deforme.
- Analyse spectrale et estimation du gain : Elle calcule la Transformée de Fourier Rapide (FFT) normalisée des deux signaux. Elle estime ensuite la valeur du gain (gain_mesure) en effectuant le rapport entre l'amplitude de la composante continue du signal déformé et celle du signal de base (S_deforme(1) / S_base(1)).

- Visualisation graphique : Si un axe d'affichage est fourni, elle trace sur le même graphique le spectre du signal déformé par le gain (en rouge) et celui du signal de base (en bleu pointillé), avec grille, titres, légendes et étiquettes d'axes.

6.4.3 GAIN + OFFSET

La fonction correcteGAIN_OFFSET génère un signal de base, lui applique une déformation combinée de type gain puis offset (choix 3), estime séparément le gain et l'offset à l'aide de l'analyse spectrale, puis trace et compare graphiquement les spectres.

```
function [s_base, s_deforme, s_corrige, gain_mesure, offset_mesure] = correcteGAIN_OFFSET(M, Ai, Fi, fe, P0, P1, ax_spectre, ax_correction)
    % . Signal de base (avec +1 pour observer le gain à 0 Hz)
    s_temp = gene_signal(M, Ai, Fi, fe, 0);
    s_base = (s_temp - mean(s_temp)) + 1;

    % . Application de la déformation GAIN + OFFSET (choix 3)
    choix_deformation = 3;
    s_deforme = deform_signal(s_base, choix_deformation, P0, P1);

    % . Calcul de la FFT
    N = length(s_deforme);
    S_deforme = abs(fft(s_deforme)) / N;
    S_base    = abs(fft(s_base)) / N;

    % . Estimation du Gain sur un pic alternatif (ex: f > 0 Hz)
    [~, idx_peak] = max(S_base(2:floor(N/2)));
    idx_peak = idx_peak + 1; % Ajustement d'indice
    gain_mesure = S_deforme(idx_peak) / S_base(idx_peak);

    % Estimation de l'Offset à 0 Hz (en retirant la contribution du gain sur le +1)
    offset_mesure = S_deforme(1) - (1 * gain_mesure);

    % . Affichage des spectres
    if nargin >= 7 && ~isempty(ax_spectre)
        N2 = floor(N/2);
        f = (0:N2-1) * fe / N;

        hold(ax_spectre, 'off');
        stem(ax_spectre, f, S_deforme(1:N2), 'r', 'LineWidth', 2, 'DisplayName', 'Signal Déformé');
        hold(ax_spectre, 'on');
        stem(ax_spectre, f + 0.1, S_base(1:N2), 'b--', 'LineWidth', 1.2, 'DisplayName', 's_i - \langle s_i \rangle + 1');

        grid(ax_spectre, 'on');
        xlabel(ax_spectre, 'Fréquence (Hz)');
        ylabel(ax_spectre, '|S(f)|');
        title(ax_spectre, 'Spectre : Gain + OFFSET');
        legend(ax_spectre, 'show');
```

Ce que fait la fonction :

- Génération et préparation du signal : Elle crée un signal temporaire via gene_signal, le centre et lui ajoute +1 pour disposer d'une composante connue à 0 Hz formant ainsi le signal de base s_base.

- Application de la déformation : Elle applique la déformation combinée (Gain + Offset, choix 3) à l'aide de la fonction deform_signal avec les paramètres P0 et P1 pour obtenir le signal déformé s_deforme.
- Analyse spectrale et estimations :

- Elle calcule la Transformée de Fourier Rapide (FFT) normalisée des deux signaux.
- Elle estime le gain (gain_mesure) en faisant le rapport des amplitudes sur un pic alternatif du spectre (f > 0 Hz) pour s'affranchir de l'offset.
- Elle estime l'offset (offset_mesure) à 0 Hz en soustrayant la contribution du gain sur la composante continue initiale du signal.

- Visualisation graphique : Si les axes d'affichage sont fournis, elle superpose le spectre du signal déformé (en rouge) et celui du signal de base (en bleu pointillé), agrémentés d'une grille, d'étiquettes d'axes, d'un titre et d'une légende.

6.4.4 OFFSET + GAIN

La fonction correcteOFFSET_GAIN génère un signal de base, lui applique une déformation combinée de type offset puis gain (choix 4), estime séparément le gain et l'offset à l'aide de l'analyse spectrale, puis trace et compare graphiquement les spectres.

Ce que fait la fonction :

- Génération et préparation du signal : Elle crée un signal temporaire via gene_signal, le centre et lui ajoute +1 pour disposer d'une composante connue à 0 Hz formant ainsi le signal de base s_base.
- Application de la déformation : Elle applique la déformation combinée (Offset + Gain, choix 4) à l'aide de la fonction deform_signal avec les paramètres P0 et P1 pour obtenir le signal déformé s_deforme.

- Analyse spectrale et estimations :
  - Elle calcule la Transformée de Fourier Rapide (FFT) normalisée des deux signaux.
  - Elle estime le gain (gain_mesure) en faisant le rapport des amplitudes sur un pic alternatif du spectre (f > 0 Hz).
  - Elle estime l'offset (offset_mesure) en divisant la composante continue du signal déformé par le gain mesuré, puis en soustrayant le +1 initial.
- Visualisation graphique : Si les axes d'affichage sont fournis, elle superpose le spectre du signal déformé (en rouge) et celui du signal de base (en bleu pointillé), agrémentés d'une grille, d'étiquettes d'axes, d'un titre et d'une légende.

```
function [s_base, s_deforme, s_corrige, gain_mesure, offset_mesure] = correcteOFFSET_GAIN(M, Ai, Fi, fe, P0, P1, ax_spectre, ax_correction)
    %  Signal de base (avec +1 pour observer la composante)
    s_temp = gene_signal(M, Ai, Fi, fe, 0);
    s_base = (s_temp - mean(s_temp)) + 1;

    %  Application de la déformation OFFSET + GAIN (choix 4)
    choix_deformation = 4;
    s_deforme = deform_signal(s_base, choix_deformation, P0, P1);

    % Calcul de la FFT
    N = length(s_deforme);
    S_deforme = abs(fft(s_deforme)) / N;
    S_base    = abs(fft(s_base)) / N;

    % Estimation du Gain sur un pic AC (f > 0 Hz)
    [~, idx_peak] = max(S_base(2:floor(N/2)));
    idx_peak = idx_peak + 1;
    gain_mesure = S_deforme(idx_peak) / S_base(idx_peak);

    % Estimation de l'Offset (l'offset a été multiplié par le gain)
    offset_mesure = (S_deforme(1) / gain_mesure) - 1;

    % Affichage des spectres
    if nargin >= 7 && ~isempty(ax_spectre)
        N2 = floor(N/2);
        f = (0:N2-1) * fe / N;

        hold(ax_spectre, 'off');
        stem(ax_spectre, f, S_deforme(1:N2), 'r', 'LineWidth', 2, 'DisplayName', 'Signal Déformé');
        hold(ax_spectre, 'on');
        stem(ax_spectre, f + 0.1, S_base(1:N2), 'b--', 'LineWidth', 1.2, 'DisplayName', 's_i - \langle s_i \rangle + 1');

        grid(ax_spectre, 'on');
        xlabel(ax_spectre, 'Fréquence (Hz)');
        ylabel(ax_spectre, '|S(f)|');
        title(ax_spectre, 'Spectre : OFFSET + Gain');
        legend(ax_spectre, 'show');
```

6.4.5 Exponentielle

La fonction correcteExpo génère un signal de base, lui applique une déformation non linéaire de type exponentiel (choix 5), estime les paramètres de déformation par régression linéaire, calcule le signal corrigé par transformation inverse, puis affiche et compare à la fois les spectres et les signaux temporels.

```
function [s_base, s_deforme, s_corrige, lambda_estime, mu_estime] = correcteExpo
    %  Génération du signal de base
    s_temp = gene_signal(M, Ai, Fi, fe, 0);
    s_base = s_temp - mean(s_temp); % Signal centré à 0

    %  Application de la déformation exponentielle (choix = 5)
    choix_deformation = 5;
    s_deforme = deform_signal(s_base, choix_deformation, lambda, mu);

    %  Estimation de Mu (Composante continue/offset minimum)
    mu_estime = min(s_deforme) - exp(lambda * min(s_base));

    % En cas de petite dérive, sécurité pour éviter log(0 ou négatif)
    s_sans_mu = s_deforme - min(s_deforme) + 1e-6;

    %  Régression Linéaire pour estimer Lambda (y = p1 * x)
    % Linéarisation : y = ln(s_deforme - mu), x = s_base
    y = log(s_sans_mu);
    x = s_base(:);
    y = y(:);

    % Ajustement linéaire du premier degré : y = p1*x + p0 avec polyfit
    p = polyfit(x, y, 1);
    lambda_estime = p(1); % Le coefficient directeur donne lambda

    % Correction (Transformation inverse : logarithme puis division par lambda)
    s_corrige = log(s_deforme - mu_estime) / lambda_estime;

    %  Affichage du Spectre / FFT (si l'axe est fourni)
    if nargin >= 7 && ~isempty(ax_spectre)
        N = length(s_deforme);
        N2 = floor(N/2);
        f = (0:N2-1) * fe / N;

        S_deforme = abs(fft(s_deforme)) / N;
        S_base    = abs(fft(s_base)) / N;

        hold(ax_spectre, 'off');
        stem(ax_spectre, f, S_deforme(1:N2), 'r', 'LineWidth', 1.5, 'DisplayName', 'Signal Déformé');
        hold(ax_spectre, 'on');
        stem(ax_spectre, f + 0.1, S_base(1:N2), 'b--', 'LineWidth', 1.2, 'DisplayName', 'Signal Original');

        grid(ax_spectre, 'on');
        xlabel(ax_spectre, 'Fréquence (Hz)');
        ylabel(ax_spectre, '|S(f)|');
        title(ax_spectre, 'Spectre : Exponentielle');
        legend(ax_spectre, 'show');
    end

    % Affichage de la Correction (Menu Correction)
    if nargin >= 8 && ~isempty(ax_correction)
        T = 1;
        t = (0 : 1/fe : T - 1/fe);

        hold(ax_correction, 'off');
        plot(ax_correction, t, s_base, 'b--', 'LineWidth', 1.2, 'DisplayName', 'Signal Origine');
        hold(ax_correction, 'on');
        plot(ax_correction, t, s_corrige, 'g', 'LineWidth', 1.5, 'DisplayName', 'Signal Corrigé');

        grid(ax_correction, 'on');
        xlabel(ax_correction, 'Temps (s)');
        ylabel(ax_correction, 'Amplitude');
        title(ax_correction, 'Correction Exponentielle');
        legend(ax_correction, 'show');
    end
```

Ce que fait la fonction :

- Génération et préparation du signal : Elle crée un signal temporaire via gene_signal, le centre et lui ajoute +1 pour disposer d'une composante connue à 0 Hz formant ainsi le signal de base s_base.
- Application de la déformation : Elle applique la déformation combinée (Offset + Gain, choix 4) à l'aide de la fonction deform_signal avec les paramètres P0 et P1 pour obtenir le signal déformé s_deforme.
- Analyse spectrale et estimations :
  - Elle calcule la Transformée de Fourier Rapide (FFT) normalisée des deux signaux.
  - Elle estime le gain (gain_mesure) en faisant le rapport des amplitudes sur un pic alternatif du spectre (f > 0 Hz).
  - Elle estime l'offset (offset_mesure) en divisant la composante continue du signal déformé par le gain mesuré, puis en soustrayant le +1 initial.
- Visualisation graphique : Si les axes d'affichage sont fournis, elle superpose le spectre du signal déformé (en rouge) et celui du signal de base (en bleu pointillé), agrémentés d'une grille, d'étiquettes d'axes, d'un titre et d'une légende.

6.4.6 Logarithme

La fonction correcteLogarithmique génère un signal de base, lui applique une déformation non linéaire de type logarithmique (choix 6), estime les paramètres de déformation par linéarisation et régression linéaire, calcule le signal corrigé par transformation inverse, puis affiche et compare à la fois les spectres et les signaux temporels.

```
function [s_base, s_deforme, s_corrige, lambda_estime, mu_estime] = correcteLogar
    % Génération du signal de base
    s_temp = gene_signal(M, Ai, Fi, fe, 0);
    s_base = s_temp - mean(s_temp); % Signal centré à 0

    % Application de la déformation logarithmique (choix = 6)
    choix_deformation = 6;
    s_deforme = deform_signal(s_base, choix_deformation, lambda, mu);

    %  Estimation de Mu (Offset / composante continue)
    % Quand s_base = 0, log(abs(0)+1) = 0, donc s_deforme vaut mu
    mu_estime = min(s_deforme);

    %  Régression Linéaire pour estimer Lambda (y = p1 * x)
    % Linéarisation : y = exp(s_deforme - mu) - 1, x = |s_base|
    y = exp(s_deforme - mu_estime) - 1;
    x = abs(s_base(:));
    y = y(:);

    % Ajustement linéaire du premier degré : y = p1*x + p0 avec polyfit
    p = polyfit(x, y, 1);
    lambda_estime = p(1); % Le coefficient directeur donne lambda

    %  Correction (Transformation inverse)
    % Recréation du signal en préservant le signe d'origine
    s_corrige = sign(s_base) .* (exp(s_deforme - mu_estime) - 1) / lambda_estime;

    %  Affichage du Spectre / FFT (si l'axe est fourni)
    if nargin >= 7 && ~isempty(ax_spectre)
        N = length(s_deforme);
        N2 = floor(N/2);
        f = (0:N2-1) * fe / N;

        S_deforme = abs(fft(s_deforme)) / N;
        S_base    = abs(fft(s_base)) / N;

```

```
        hold(ax_spectre, 'off');
        stem(ax_spectre, f, S_deforme(1:N2), 'r', 'LineWidth', 1.5, 'DisplayName', 'Signal Déformé');
        hold(ax_spectre, 'on');
        stem(ax_spectre, f + 0.1, S_base(1:N2), 'b--', 'LineWidth', 1.2, 'DisplayName', 'Signal Original');

        grid(ax_spectre, 'on');
        xlabel(ax_spectre, 'Fréquence (Hz)');
        ylabel(ax_spectre, '|S(f)|');
        title(ax_spectre, 'Spectre : Logarithmique');
        legend(ax_spectre, 'show');
    end

    % Affichage de la Correction (Menu Correction)
    if nargin >= 8 && ~isempty(ax_correction)
        T = 1;
        t = (0 : 1/fe : T - 1/fe);

        hold(ax_correction, 'off');
        plot(ax_correction, t, s_base, 'b--', 'LineWidth', 1.2, 'DisplayName', 'Signal Origine');
        hold(ax_correction, 'on');
        plot(ax_correction, t, s_corrige, 'g', 'LineWidth', 1.5, 'DisplayName', 'Signal Corrigé');

        grid(ax_correction, 'on');
        xlabel(ax_correction, 'Temps (s)');
        ylabel(ax_correction, 'Amplitude');
        title(ax_correction, 'Correction Logarithmique');
        legend(ax_correction, 'show');
    end
end
```

Ce que fait concrètement cette fonction :

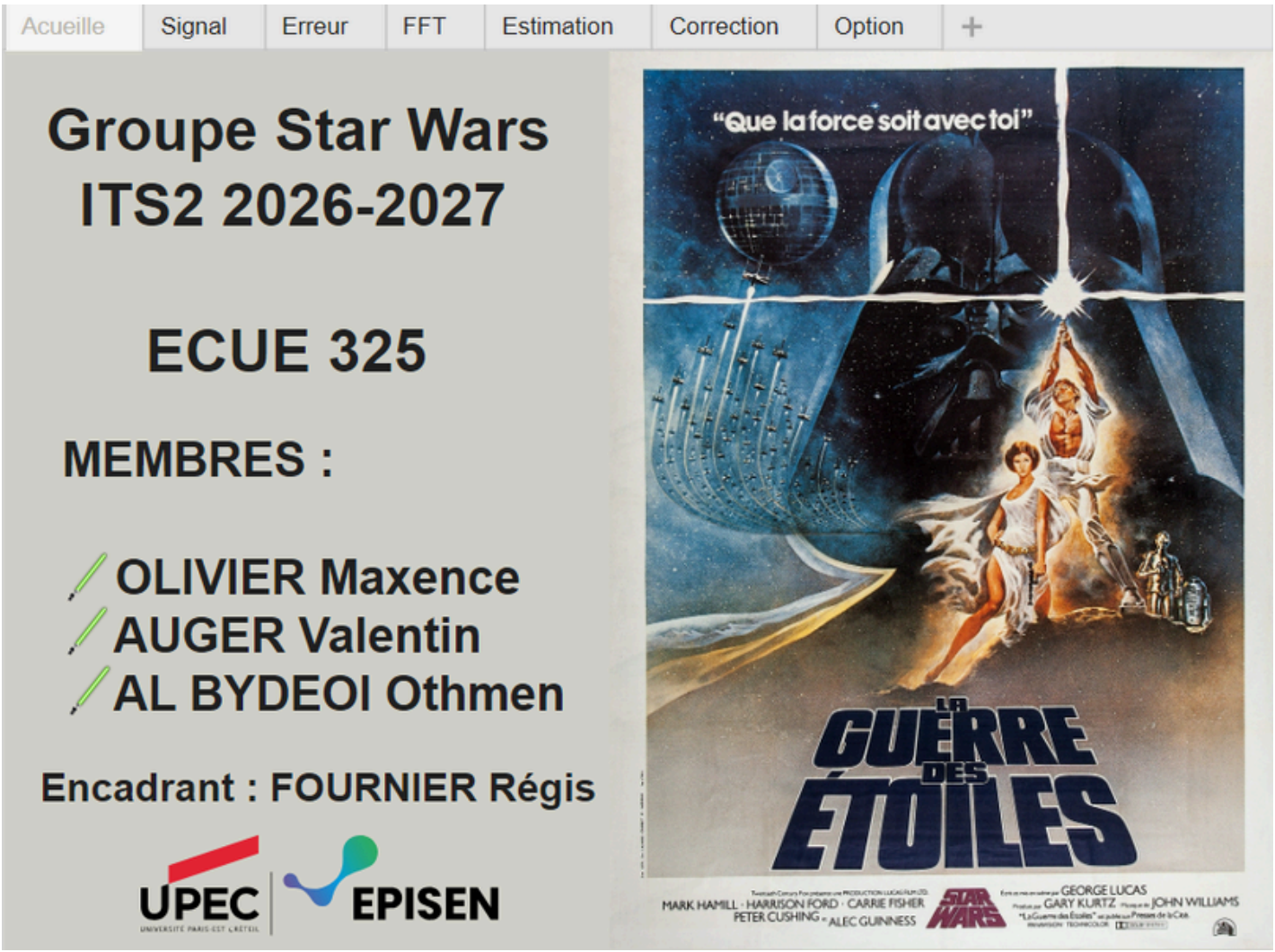
- Génération et centrage : Elle crée un signal de base s_base à l'aide de gene_signal puis le centre avec une moyenne nulle.
- Application de la déformation : Elle applique la déformation logarithmique via deform_signal (choix 6) avec les paramètres lambda et mu pour obtenir le signal déformé s_deforme.
- Estimation et correction (Linéarisation) :
  - Elle estime le paramètre mu_estime en se basant sur le minimum du signal déformé.
  - Elle applique une transformation exponentielle pour linéariser le logarithme et utilise la fonction polyfit (régression linéaire du premier degré) afin d'estimer le coefficient directeur qui correspond à lambda_estime.
  - Elle calcule le signal corrigé s_corrige en appliquant la transformation inverse tout en préservant le signe d'origine du signal.

7. IHM

Pour la réalisation de l'IHM (Interface Homme-Machine), nous avons utilisé App Designer sous MATLAB (version 2025b). L'interface est structurée en plusieurs pages à l'aide du composant Tab Group.

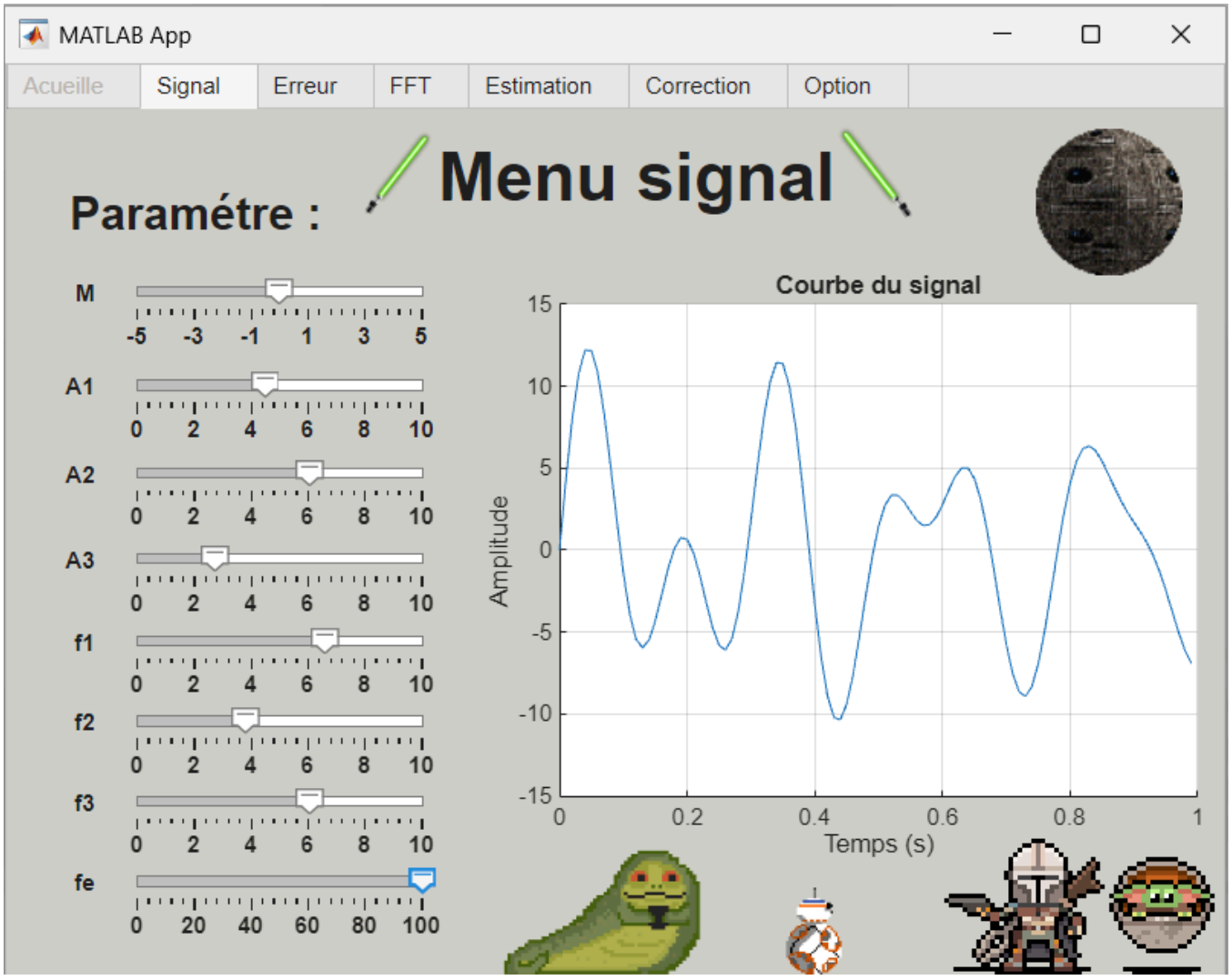
7.1. Page d'accueil

Une page d'accueil a été conçue afin de soigner l'aspect visuel de l'interface. Lors de l'accès à cette page, la bande-son de Star Wars est automatiquement déclenchée.



D'un point de vue programmation, la gestion audio s'appuie sur une Private Property déclarée dans Code View sous le nom de variable Player. Celle-ci stocke l'instance du lecteur audio, qui est ensuite sollicitée par une fonction dédiée pour exécuter la lecture du fichier audio au format MPEG.

7.2. Page Menu Signal



Pour moduler le signal généré, plusieurs curseurs (Sliders) ont été mis en place avec des plages de valeurs spécifiques :

- **Valeur moyenne / offset (M)** : [-5 ; 5]
  Permet de décaler le signal vers le haut ou vers le bas autour de zéro (composante continue). L'intervalle de [-5 à 5] offre un centrage symétrique adapté aux amplitudes testées.

- **Amplitudes (A1, A2, A3)** : [0 ; 10]
  Définissent les amplitudes respectives des différentes harmoniques / composantes du signal. L'intervalle de 0 à 10 évite d'avoir des valeurs négatives tout en permettant de faire varier l'importance de chaque composante jusqu'à une valeur maximale raisonnable.

- **Fréquences des composantes (f1, f2, f3)** : [0 ;10 Hz]
  Règlent les fréquences fondamentales et harmoniques du signal dans une plage de basse fréquence (0 à 10 Hz), idéale pour l'observation visuelle et l'analyse du signal sur le graphique.

- **Fréquence d'échantillonnage (fe)** : [0 ;100 Hz]
  Définit le rythme d'échantillonnage de la fonction. La valeur maximale (100 Hz) est choisie pour respecter largement le théorème de Shannon-Nyquist (fe ≥ 2 fmax) par rapport à la fréquence maximale autorisée (10 Hz), garantissant ainsi une bonne reconstitution sans repliement de spectre.

Pour la liaison entre l'interface (UIAxes) et le script MATLAB gene-signal
Afin d'afficher en temps réel le signal calculé :

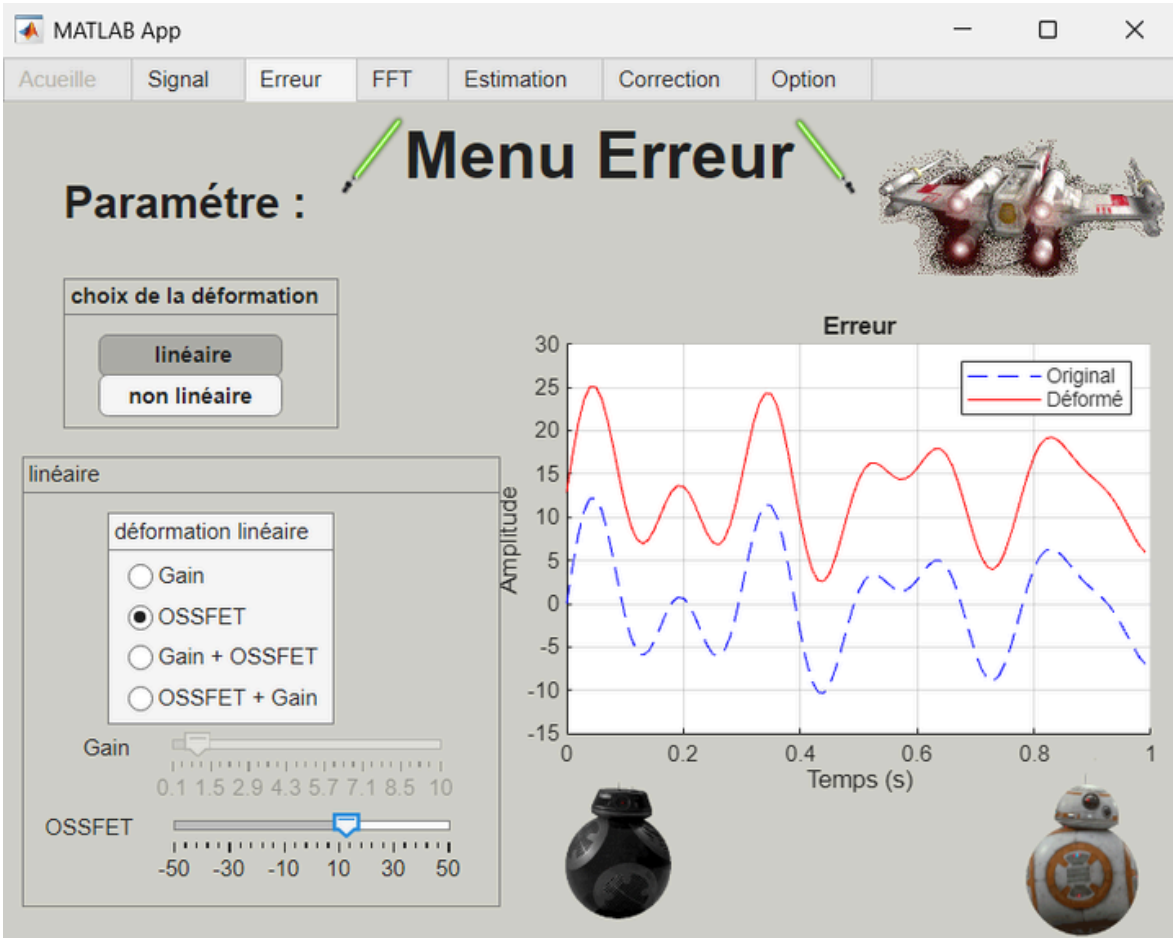
- Chaque modification d'un curseur déclenche une fonction de rappel (Callback).

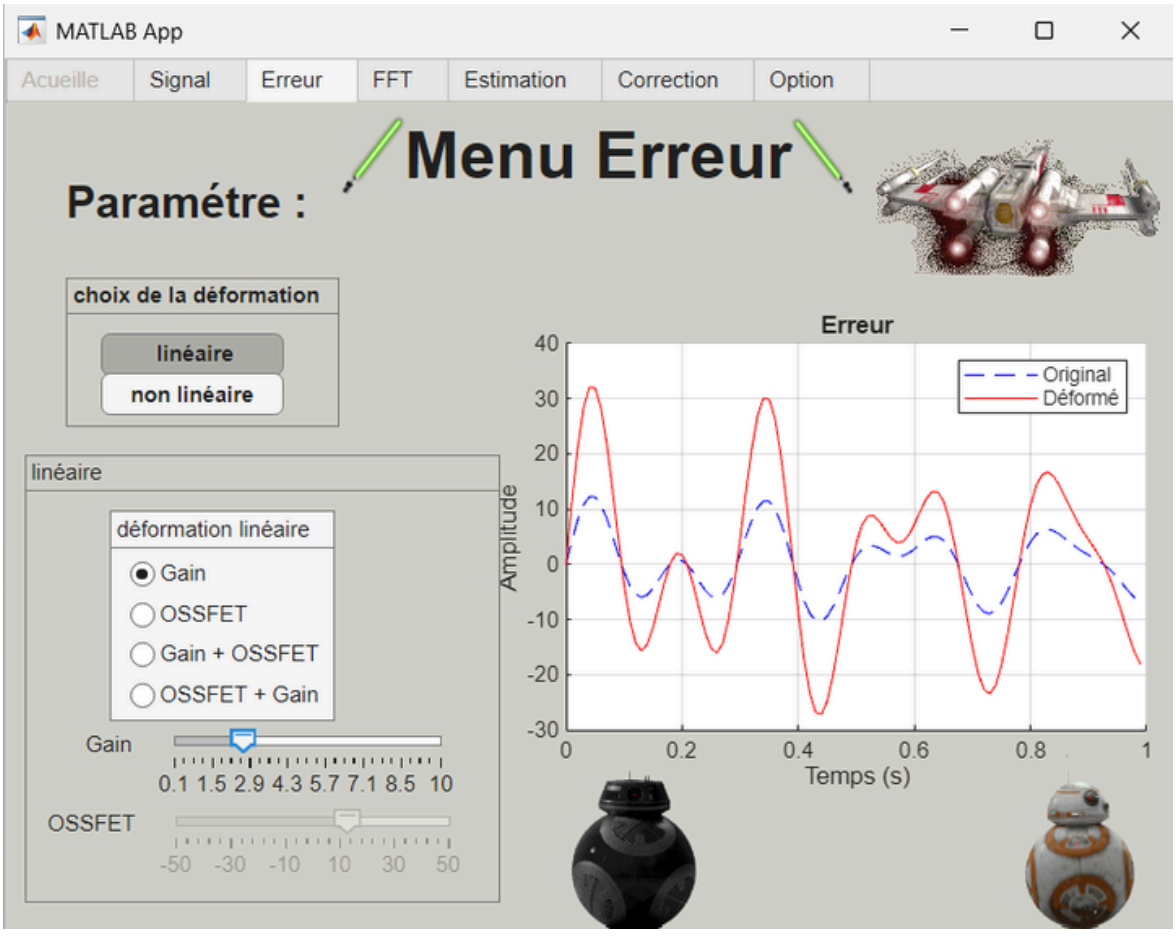
- Ce callback récupère les valeurs courantes des paramètres (M, A1, A2, A3, f1, f2, f3, fe) et les transmet en arguments au script/fonction gene-signal.

- Le script gene-signal génère le vecteur temps t ainsi que le vecteur signal correspondant.

- La courbe est directement affichée sur le composant graphique de l'IHM (app.UIAxes) à l'aide de la commande plot(app.UIAxes, t, signal).

7.3. Page Menu Erreur





Le Menu Erreur permet d'appliquer et de tester différentes déformations (linéaires et non linéaires) sur le signal de base afin d'analyser leur impact et de préparer la phase de correction.

Plages de valeurs des curseurs

Pour configurer les paramètres de déformation, l'interface intègre plusieurs curseurs adaptés selon le mode choisi :

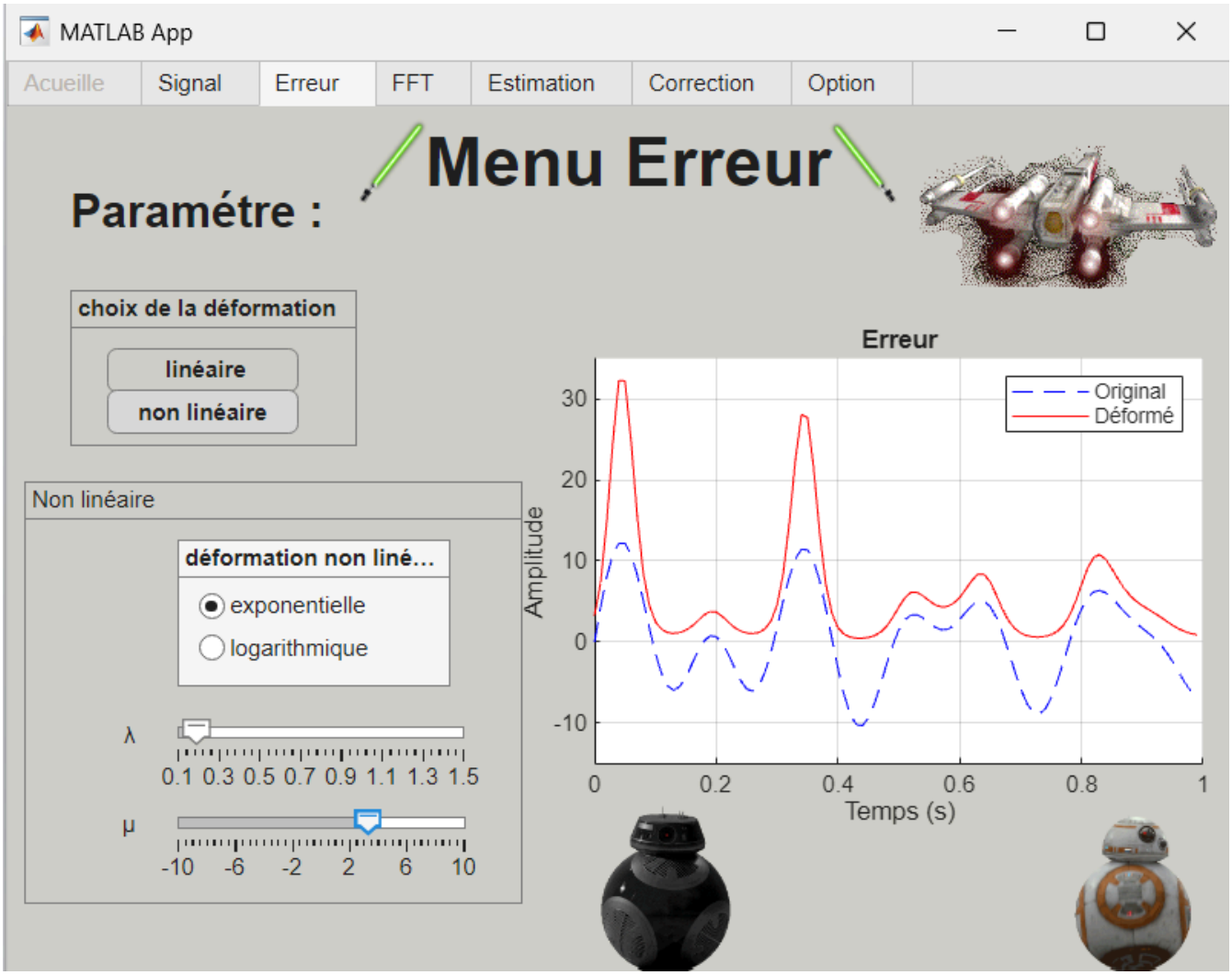
- Mode Linéaire :
  - Curseur Gain : Plage allant de 0.1 à 10. Il permet de modérer l'amplification ou l'atténuation du signal sans introduire de valeurs négatives excessives.
  - Curseur Offset : Plage allant de -50 à 50. Il permet d'ajouter une composante continue importante pour décaler verticalement le signal vers le haut ou vers le bas.

Gestion dynamique de l'interface : Grisement des curseurs

Pour éviter toute incohérence lors de la manipulation, l'IHM gère dynamiquement l'état des curseurs en fonction de la déformation sélectionnée :

- Lorsque l'utilisateur sélectionne un mode unitaire (par exemple Gain seul), le curseur Offset est automatiquement désactivé (grisé) et inversement.

- Pourquoi ? Cela garantit que l'utilisateur ne modifie que le paramètre pertinent pour la déformation active, évitant ainsi les erreurs de manipulation ou les conflits de calcul dans le script de déformation. Lorsque les modes combinés (Gain + Offset ou Offset + Gain) sont sélectionnés, les deux curseurs redeviennent actifs simultanément.



Mode Non Linéaire :

- Curseur λ : Plage allant de 0.1 à 1.5. Il contrôle le taux de croissance ou d'atténuation de la transformation non linéaire (exponentielle ou logarithmique).
- Curseur μ : Plage allant de -10 à 10. Il gère le décalage ou l'offset associé à la fonction non linéaire.

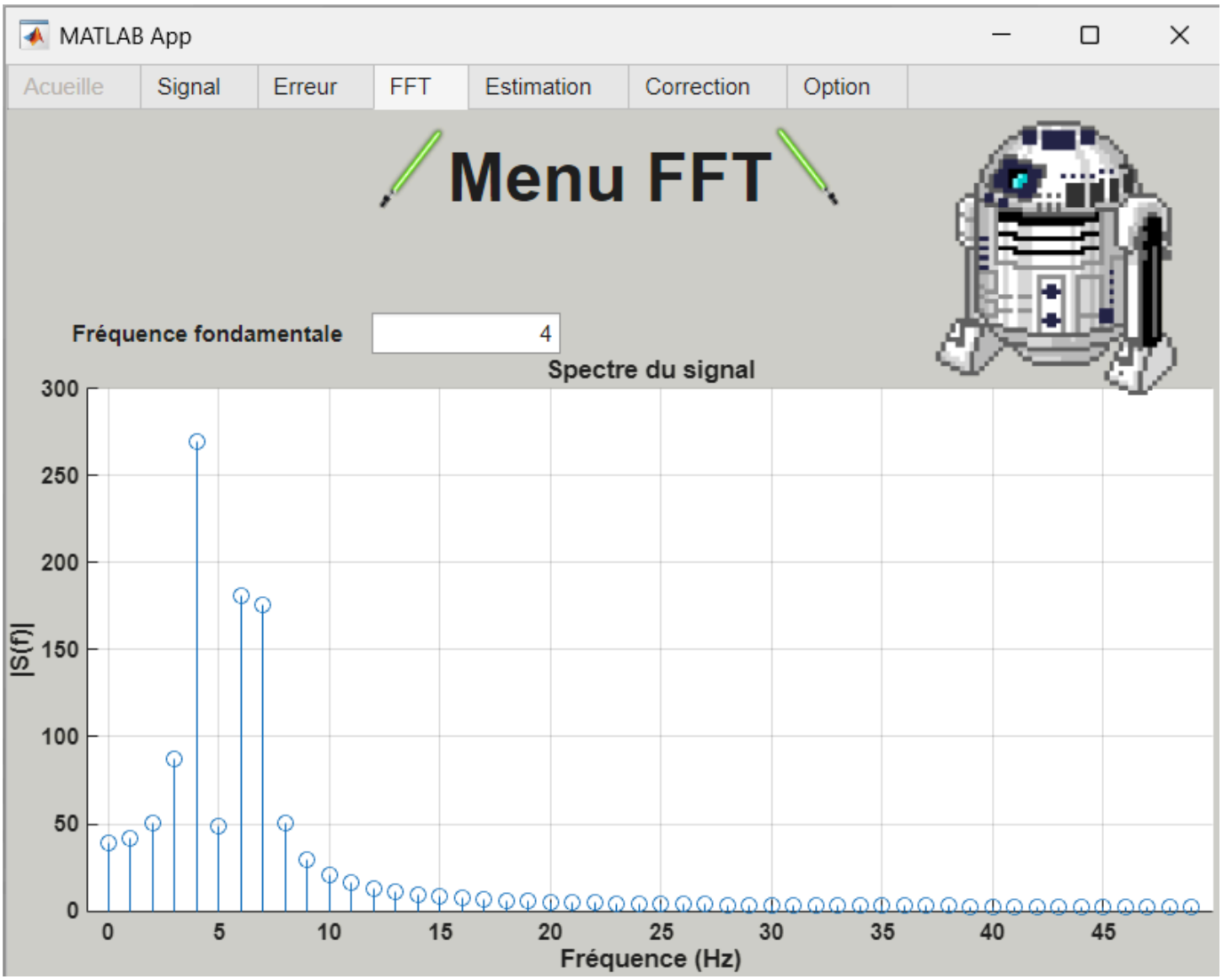
Justification du nom des paramètres non linéaires (λ , μ)

Dans la partie non linéaire, les paramètres de réglage ont été nommés λ et μ à la place de dénominations génériques (comme P0 ou P1) pour plusieurs raisons :

- Rigueur mathématique et convention : En traitement du signal et en mathématiques appliquées, les lettres grecques λ et μ sont les notations universellement consacrées pour représenter respectivement les facteurs d'échelle/taux non linéaires λ et les paramètres de translation ou de seuil μ

- Clarté : Cela permet d'identifier immédiatement la nature non linéaire de ces coefficients par rapport aux gains et offsets classiques du mode linéaire, rendant le code et l'IHM plus professionnels et cohérents avec les formules théoriques implémentées.

7.4. Page Menu FFT



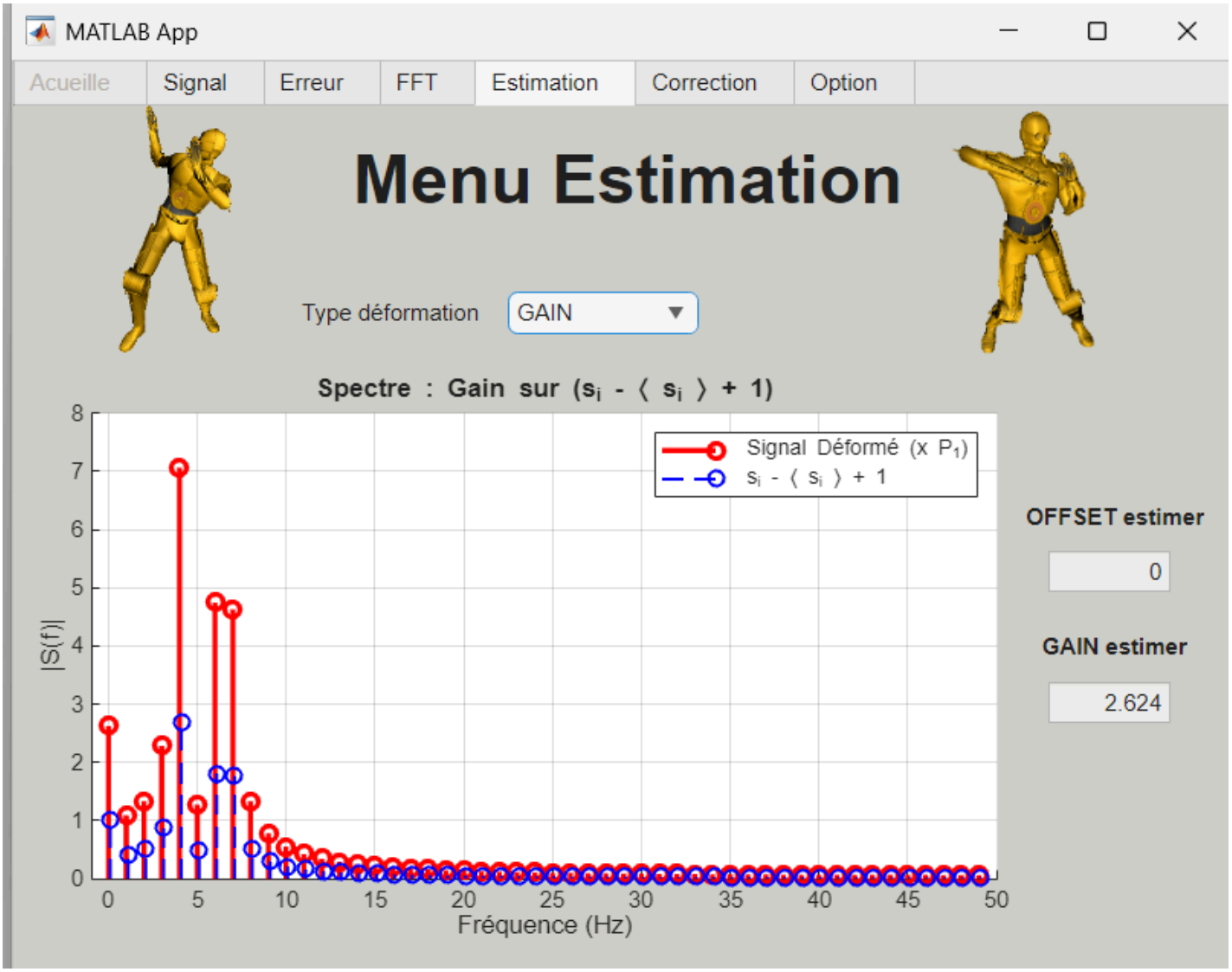
Graphique du Spectre (Spectre du signal) :

- Un composant graphique (UIAxes) affiche le spectre d'amplitude en fonction de la fréquence en Hertz (|S(f)| en fonction de Fréquence (Hz)).

- L'utilisation de la fonction stem permet de représenter sous forme de pics discrets les différentes harmoniques du signal, ainsi que la composante continue située à 0 Hz.

7.5. Page Menu Estimation

Le Menu Estimation permet de visualiser et d'évaluer quantitativement l'impact de la déformation choisie en comparant le spectre du signal de base avec celui du signal déformé.



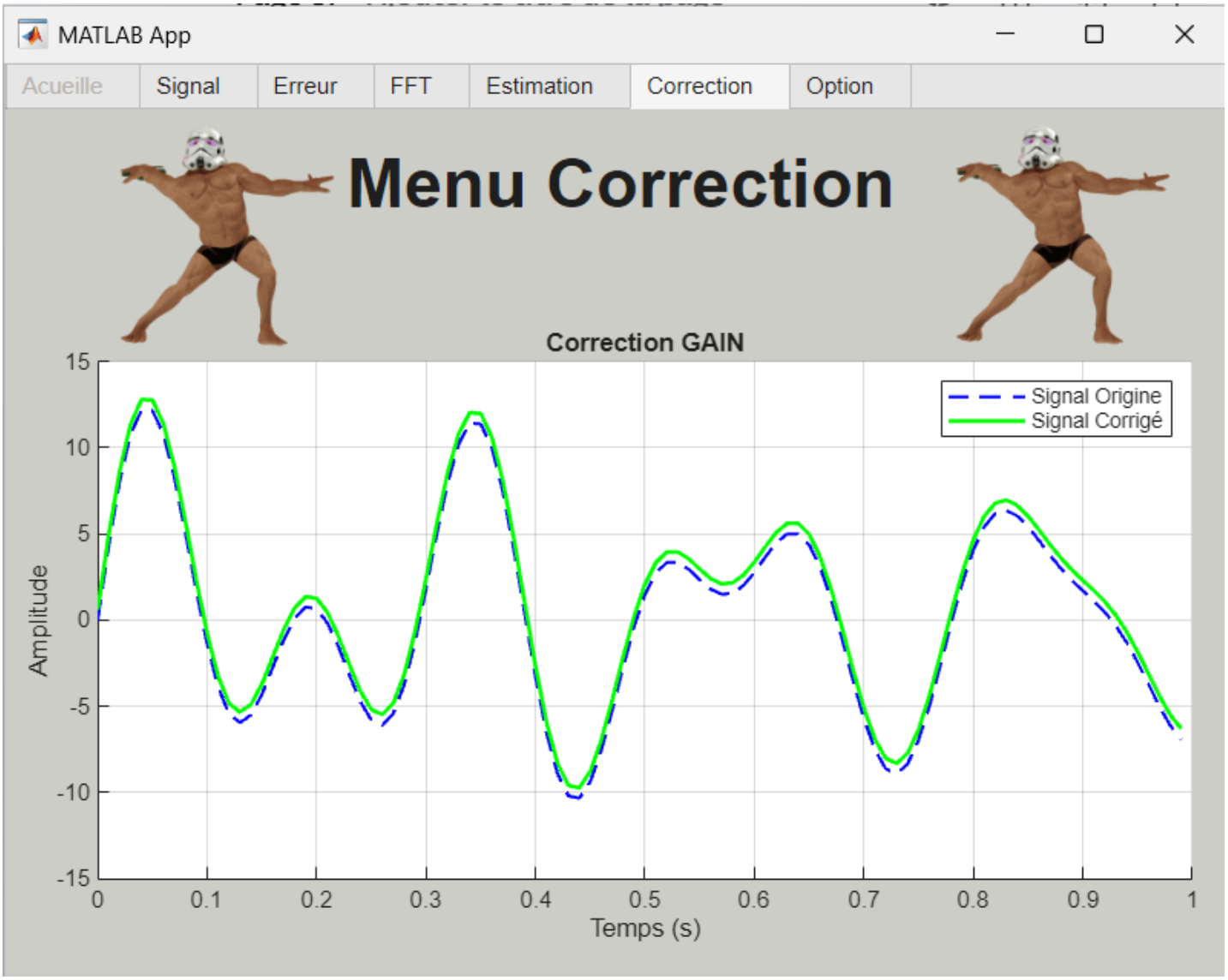
Composants et fonctionnement de l'interface

- Menu déroulant (Type déformation) :
  - Permet à l'utilisateur de sélectionner le type de déformation à analyser

- Graphique comparatif des spectres :
  - Un composant graphique (UIAxes) superpose le spectre du signal déformé (en rouge) et celui du signal de référence (en bleu pointillé).

- Champs d'estimation numérique :
  - OFFSET estimé : Affiche la valeur mesurée de l'offset calculée à partir de la composante continue du spectre.
  - GAIN estimé : Affiche la valeur mesurée du gain (par exemple, 2.624 sur l'interface) obtenue par le rapport des amplitudes entre les signaux.

7.6. Page Menu Correction



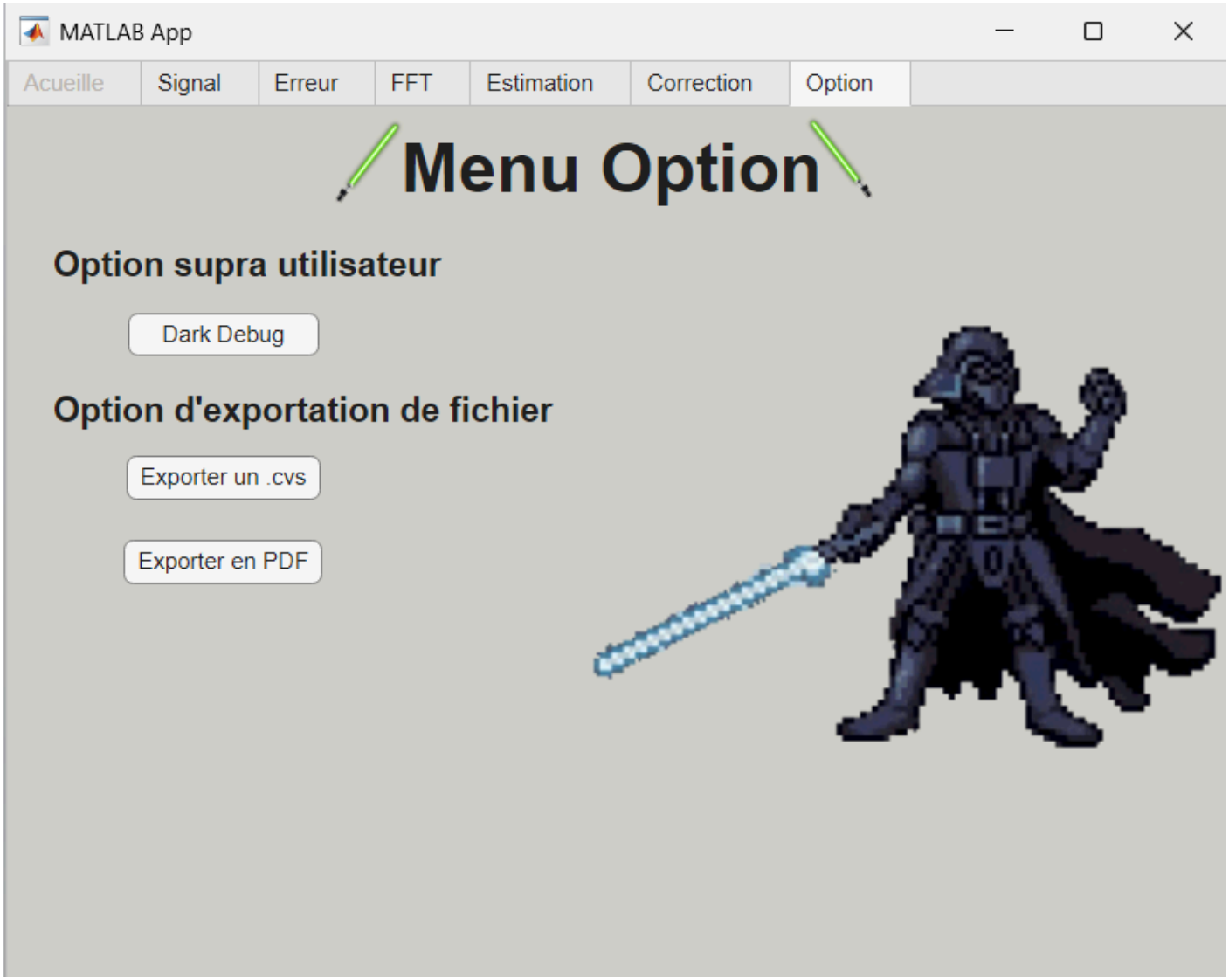
Le Menu Correction permet de valider visuellement l'efficacité des traitements de correction appliqués au signal déformé, en comparant directement dans le domaine temporel le signal d'origine et le signal restitué

Interprétation visuelle :

- La superposition quasi parfaite des deux courbes (en vert et bleu pointillé) permet à l'utilisateur de vérifier instantanément que la correction a bien supprimé la déformation subie, reconstruisant fidèlement le signal d'origine.

8.Bonus

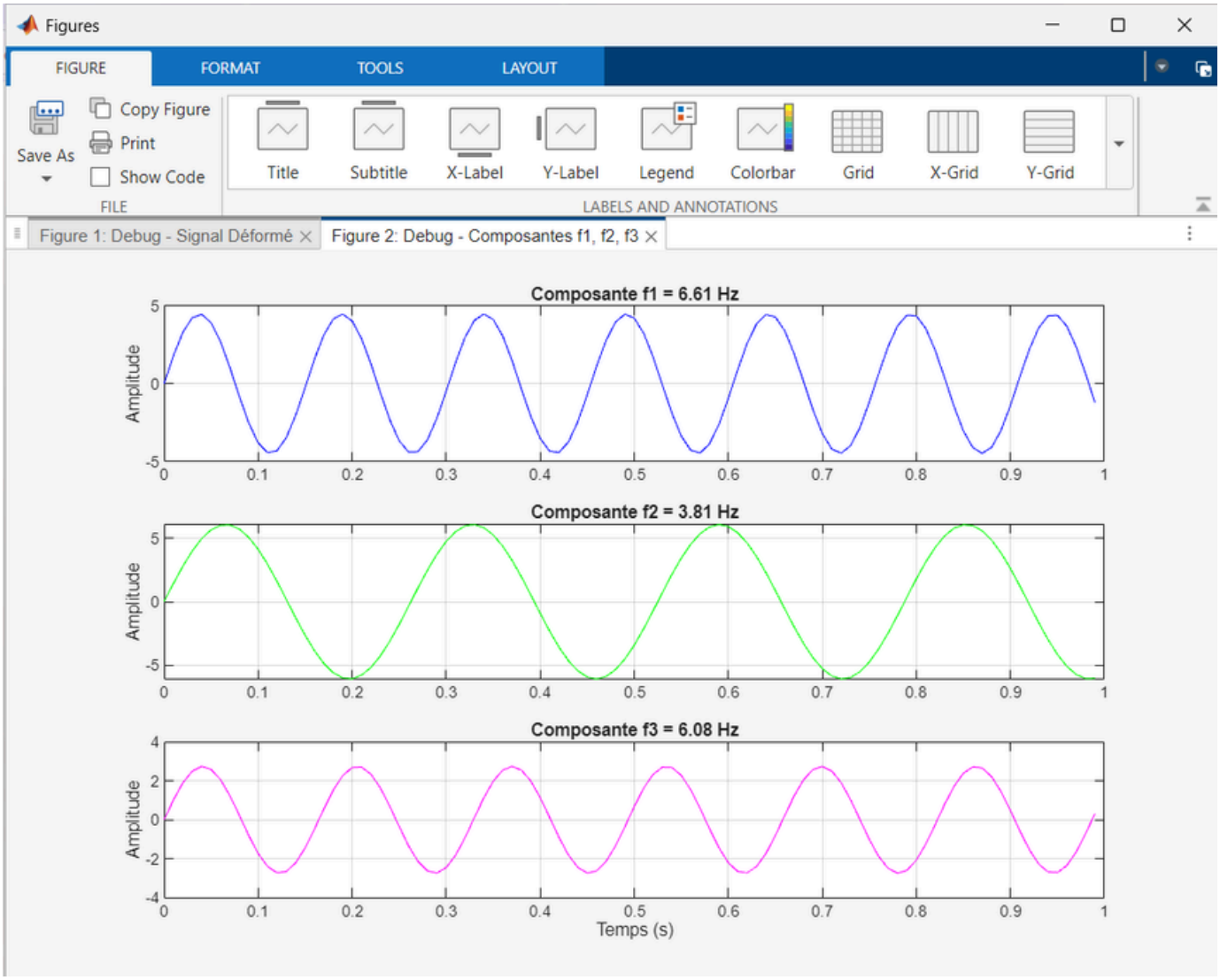
8.1. Page Menu Option



Le Menu Option regroupe les fonctionnalités avancées de débogage pour les utilisateurs experts ainsi que les outils d'exportation des données de l'application.

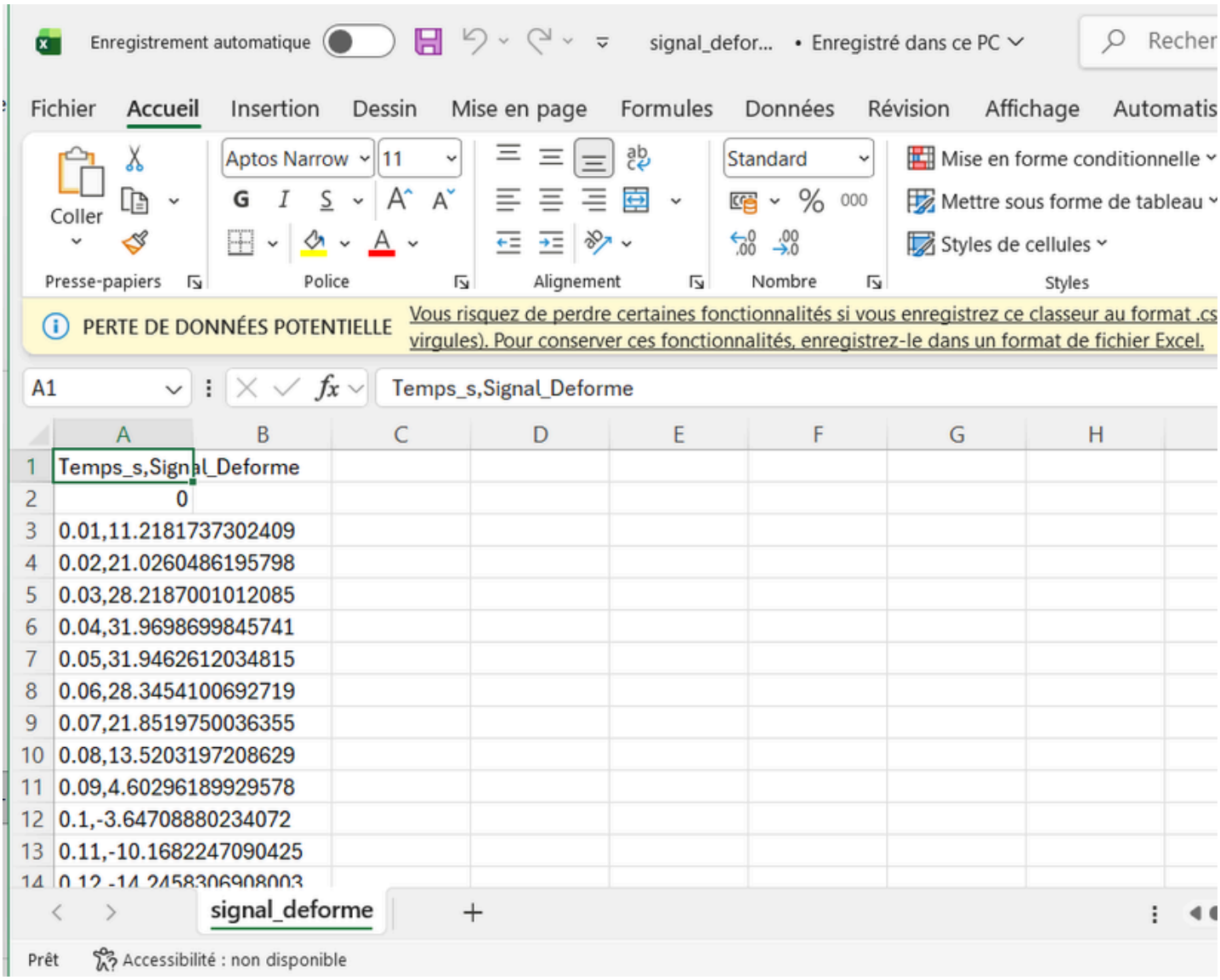
Bouton Dark Debug :

- Permet d'afficher des graphiques de contrôle détaillés (comme l'analyse séparée des composantes du signal ou des états internes) dans des fenêtres de figures indépendantes MATLAB



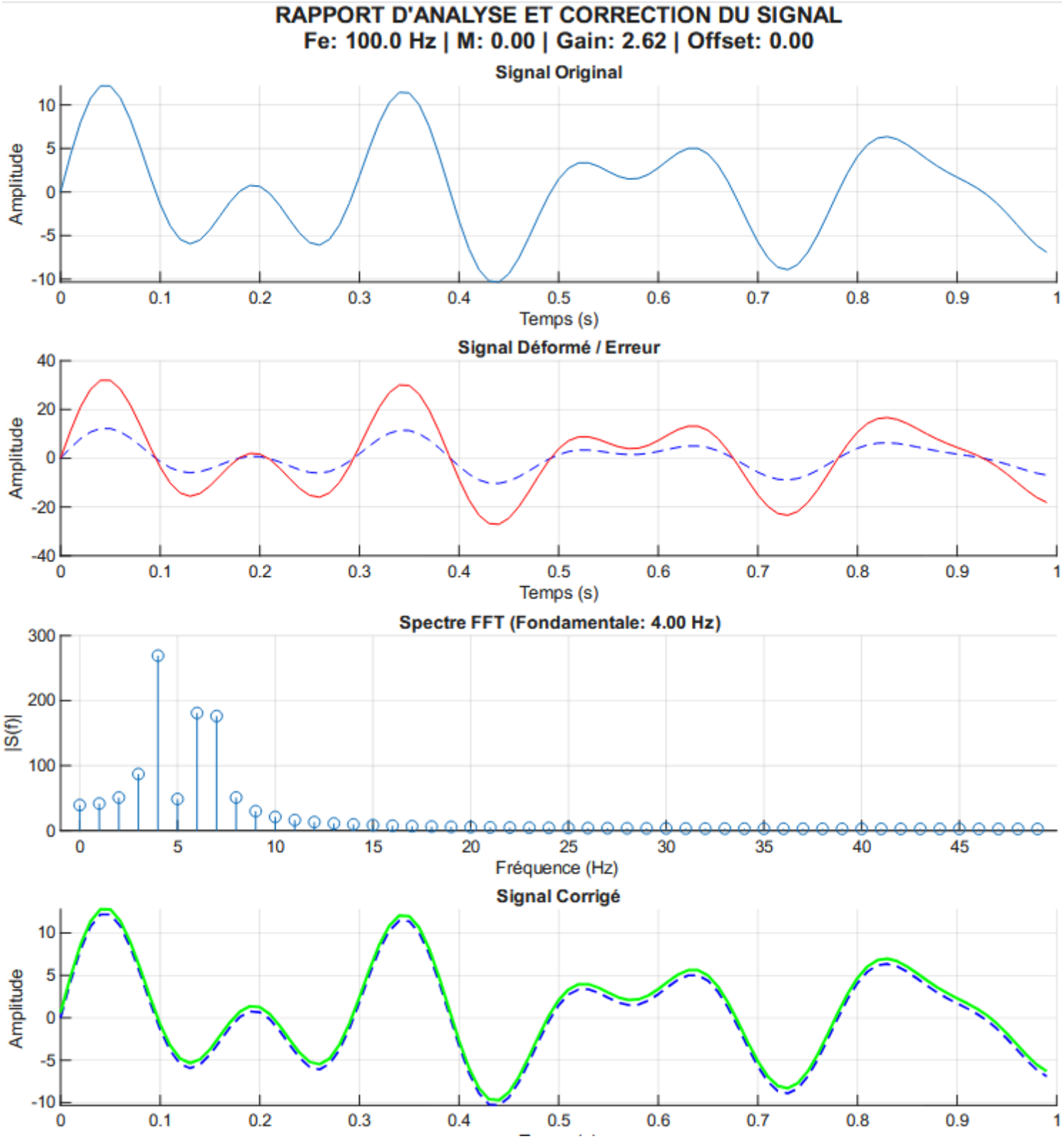
Section "Option d'exportation de fichier" :

- Bouton Exporter un .csv :
  - Permet d'exporter les données numériques des signaux (temps et amplitudes) sous forme de fichier tabulaire CSV pour une exploitation externe (par exemple sous Excel)



Bouton Exporter en PDF :

- Permet d'extraire et d'enregistrer un rapport complet d'analyse, incluant les graphiques récapitulatifs du signal original, du signal déformé, du spectre FFT et du signal corrigé, sous format document PDF


RAPPORT D'ANALYSE ET CORRECTION DU SIGNAL
Fe: 100.0 Hz | M: 0.00 | Gain: 2.62 | Offset: 0.00
Signal Original
Amplitude
Temps (s)
Signal Déformé / Erreur
Amplitude
Temps (s)
Spectre FFT (Fondamentale: 4.00 Hz)
|S(f)|
Fréquence (Hz)
Signal Corrigé
Amplitude

9.Conclusion

Ce projet, réalisé dans le cadre de l'ECUE 325 — Systèmes de mesures nomades, a permis de concevoir et d'implémenter avec succès une chaîne complète de simulation, d'analyse et de correction de signaux sous MATLAB.

En partant de la modélisation théorique des capteurs et de leur chaîne d'acquisition, nous avons développé des scripts algorithmiques robustes permettant de générer des signaux complexes, d'y injecter diverses perturbations (erreurs linéaires de gain et d'offset, ainsi que des déformations non linéaires de type exponentiel et logarithmique), puis de les exploiter à l'aide de la Transformée de Fourier Rapide (FFT).

L'utilisation conjointe du domaine fréquentiel et temporel a rendu possible l'estimation précise des paramètres d'erreur, garantissant une restauration fidèle du signal d'origine. Par ailleurs, l'intégration de l'ensemble de ces fonctionnalités au sein d'une Interface Homme-Machine (IHM) ergonomique et interactive, développée via MATLAB App Designer, confère à cet outil une véritable dimension pédagogique et opérationnelle.

Enfin, l'ajout de fonctionnalités avancées — telles que l'ambiance sonore immersive, les outils de débogage (« Dark Debug ») et les options d'exportation de données (CSV et rapports PDF automatisés) — témoigne de la rigueur technique et de l'aboutissement de ce travail de groupe. Ce projet constitue ainsi une application concrète et complète des concepts fondamentaux du traitement du signal et de l'ingénierie des interfaces graphiques.

Documentation annexe

Documentation qui a servi à coder ce projet.

https://fr.mathworks.com/help/matlab/ref/appdesigner.html
https://fr.mathworks.com/matlabcentral/fileexchange/
https://stackoverflow.com/questions/tagged/matlab